COMSATS University Islamabad,
Sahiwal Pakistan

FYP Portal

By

Ammr Wahab	**CIIT/FA21-BSE-101/SWL**
M Mubashar Zafar Bhatti	**CIIT/FA21-BSE-128/SWL**

Supervisor

Dr. Javed Ferzund

Bachelor of Science in Computer Science (2021-2025)

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely based on our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other. We will stand by the consequences. No Portion of the work presented has been submitted of any application for any other degree or qualification of this or any other university or institute of learning.

Ammr Wahab **M Mubashar Zafar Bhatti**

CERTIFICATE OF APPROVAL

It is to certify that the final year project of BS (SE) "**FYP Portal**" was developed by **Ammr Wahab (CIIT/FA21-BSE-101/SWL)** and **M Mubashar Zafar Bhatti (CIIT/FA21- BSE-128/SWL)** under the supervision of **"Dr. Javed Ferzund"** and that in his opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in Software Engineering.

Supervisor
Dr. Javed Ferzund
Associate Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

External Examiner
Dr. Ali Samad
Assistant Professor
Department of Data Science
FoC, IUB

Head of Department
Dr. Zafar Iqbal Roy
Assistant Professor
Department of Computer Science
COMSATS University Islamabad, Sahiwal Campus

Executive Summary

This project presents the development of a comprehensive FYP Portal, a web-based management system designed to streamline and automate the entire lifecycle of Final Year Projects within academic institutions. Traditional FYP processes, such as group formation, supervisor allocation, document submission, progress tracking, and evaluation, are typically handled manually through emails, physical submissions, and scattered communication channels. These outdated methods often lead to inefficiencies, miscommunication, data loss, delays, and difficulty maintaining academic transparency. The FYP Portal addresses these challenges by providing a centralized, secure, and interactive digital platform for students, supervisors, and departmental administrators.

The system enables students to create groups, request supervisors, submit proposals and progress reports, track deadlines, and communicate seamlessly with faculty members. Supervisors can review submissions, provide feedback, approve or reject requests, monitor group progress, and communicate with their assigned students through dedicated tools. For administrators, the portal offers complete oversight of departmental FYP activities, including user account management, project allocation monitoring, announcement publishing, and enforcement of FYP-related policies such as group size limits and submission deadlines.

Developed using a modern web technology stack, including HTML, CSS, JavaScript (or React), PHP/Laravel (or Node), and MySQL/SQL database, the portal ensures system reliability, role-based security, responsive design, and efficient data handling. The architecture supports scalability, allowing future integration with institutional LMS/ERP systems and extended functionalities like automated evaluation or plagiarism detection.

Testing and validation confirm that the platform meets its functional and non-functional requirements, including usability, performance, security, and maintainability. The portal significantly enhances the workflow of FYP management by reducing manual burden, improving communication, increasing transparency, and ensuring timely completion of project milestones. Ultimately, the FYP Portal contributes meaningfully to academic process modernization by delivering an efficient, centralized, and future-ready management platform.

Acknowledgement

All praise is to almighty Allah who bestowed upon us a minute portion of his boundless knowledge by which we were able to accomplish this challenging task.

We are greatly indebted to our project supervisor **"Dr. Javed Ferzund"**. Without his supervision, advice, and valuable guidance, the completion of this project would have been doubtful. We are deeply indebted to them for their encouragement and continual help during this work.

And we are also thankful to our parents and family who have been a constant source of encouragement for us and brought us the values of honesty & hard work.

Ammr Wahab **M Mubashar Zafar Bhatti**

Abbreviations

FYP	Final Year Project
UI	User Interface
UX	User Experience
DB	Database
SQL	Structured Query Language
ERD	Entity Relationship Diagram
SDLC	Software Development Life Cycle
SRS	Software Requirement Specification
CRUD	Create, Read, Update, Delete
API	Application Programming Interface
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
CSS	Cascading Style Sheets
JS	JavaScript
PHP	Hypertext Pre-processor
MVC	Model View Controller
HOD	Head of Department
OTP	One-Time Password
UML	Unified Modelling Language
IDE	Integrated Development Environment

Table of Contents

List of Figures

List of Tables

Chapter 1 Introduction

1 Introduction

Final Year Projects (FYPs) are a fundamental academic requirement in undergraduate computing programs, serving as a capstone through which students apply theoretical knowledge, technical skills, and collaborative practices. These projects simulate real-world software development environments where structured planning, supervision, documentation, and evaluation are essential. However, despite the importance of FYPs, many institutions continue to manage them using outdated and fragmented processes that lack efficiency and transparency.

Traditional FYP management relies on emails, messaging applications, spreadsheets, and manual record keeping. Such disconnected methods create challenges for all stakeholders involved. Students often face confusion regarding deadlines, submissions, and supervisor feedback, while supervisors struggle to monitor multiple groups and track progress consistently. Similarly, Heads of Department lack a centralized mechanism to oversee ongoing projects and balance supervisory workloads, and administrators depend on manual processes that are prone to errors and data inconsistencies.

The FYP Portal was developed to address these limitations by introducing a centralized, web-based platform that streamlines the entire FYP management process. The system provides role-based access for students, supervisors, HODs, and administrators, enabling structured workflows for group formation, supervisor assignment, document submission, progress tracking, and official communication. All actions within the system are authenticated, validated, and securely stored, ensuring accountability and transparency throughout the project lifecycle.

By automating academic rules such as group size limits, supervision capacity, and submission validation, the FYP Portal eliminates ambiguities commonly found in manual processes. It enhances document management through organized submissions and version tracking, supports efficient supervision through centralized dashboards, and improves departmental oversight through real-time visibility. Overall, the portal modernizes FYP management by reducing administrative effort, improving coordination, and providing a reliable digital infrastructure aligned with contemporary academic and software engineering practices.

1.1 Brief

The FYP Portal is a comprehensive web-based system designed to modernize and streamline the Final Year Project management workflow within academic institutions. Traditional FYP processes often rely on scattered communication channels such as emails, messaging applications, and manually maintained documents, making it difficult for stakeholders to maintain clarity and consistency. This portal addresses those limitations by introducing a centralized platform where students, supervisors, HODs, and administrators interact through structured and well-defined interfaces. The system establishes a unified digital environment that enhances coordination, traceability, and operational efficiency throughout the entire FYP lifecycle.

At the core of the portal is a role-based access mechanism, ensuring that each user only interacts with features relevant to their responsibilities. Students can create or join groups, submit project documents, request supervisors, and receive feedback in a timely and organized manner. Supervisors, in turn, gain access to dashboards that consolidate group requests, submission

histories, and communication channels, making it easier to manage multiple groups simultaneously.[4] The HOD receives department-wide oversight capabilities, including the ability to review all groups, monitor supervisor loads, and publish official templates or announcements. Administrators retain complete control over system configurations, user creation, and institutional policy enforcement.

A distinguishing feature of this system is the inclusion of a Validation and Limits Engine, designed to automatically enforce rules such as maximum group sizes, supervisor capacity limits, and duplicate request prevention. These constraints, which are typically managed manually in traditional settings, are now integrated into the system's backend logic to reduce administrative workload and eliminate common errors. By automating these validations, the portal ensures fairness, consistency, and transparency across all stages of the FYP process.

From a technological perspective, the system utilizes React.js for the frontend, offering a dynamic and responsive user experience, while Node.js with Express.js forms the server-side backbone responsible for handling business logic, validation, and communication between users and the database. MongoDB Atlas, a cloud-hosted NoSQL database, provides secure, scalable, and high-performance storage for user data, submissions, notifications, and system configurations. Together, these technologies support real-time updates, seamless connectivity, and reliable access to information across different user roles.

In addition to its functional offerings, the FYP Portal emphasizes ease of use and accessibility. Its interface is designed to minimize cognitive load by presenting information in a clear, structured, and intuitive manner. Students unfamiliar with complex digital platforms can easily navigate menus, upload files, and monitor group activities. Supervisors benefit from organized group views and simplified feedback mechanisms, while HODs and administrators interact with tools that allow quick decision-making and transparent evaluation of the department's academic activity.

Overall, the FYP Portal stands as a holistic solution that enhances academic management through automation, standardization, and collaborative digital interaction. By tackling long-standing inefficiencies within FYP workflows, the system not only improves operational processes but also contributes to better project outcomes, timely evaluations, and enriched academic experiences for all participants. The integration of modern technologies, user-centric design, and automated validation mechanisms positions the FYP Portal as a transformative tool for contemporary academic environments.

1.2 Relevance to Course Modules

The FYP Portal incorporates concepts from multiple Software Engineering modules. Each course played a critical role in shaping the portal's architecture, interface, workflow automation, testing, and security. The following sections describe how each module directly influenced the development of the website.

1.2.1 Software Engineering

This module provided the structural, architectural, and methodological foundation for the entire project.

- **Application of Engineering Principles:** The design, architecture, and development workflow of the FYP Portal followed standard engineering principles such as modular

design, iterative SDLC models, requirement traceability, and structured documentation. These principles enabled the system to be broken into separate functional modules (Student, Supervisor, HOD, Admin), each built to be maintainable, scalable, and logically independent. The creation of SRS, SDD, UML-inspired diagrams, and design decisions all reflect learned Software Engineering methodologies.

- **Design, Planning & Lifecycle Management:** Software Engineering practices guided planning activities like feasibility study, risk identification, and task distribution across the development team. Architectural decisions, such as role-based dashboards, backend validation layers, and reusable components, were taken using engineering analysis. This ensured disciplined development, smooth integration between frontend and backend, and coherent system evolution through all SDLC phases.

1.2.2 Web Engineering

This module shaped both the frontend and backend development of the web-based portal.

- **Frontend Engineering Using React.js:** Web Engineering concepts were implemented through a component-based UI built in React.js, supporting dynamic rendering, responsive layouts, and structured navigation across dashboards for Students, Supervisors, HOD, and Admin. State management tools like React hooks enabled real-time updates to group status, supervisor availability, and notifications. Event-driven design ensured that user actions, such as sending requests or uploading documents, triggered predictable, efficient UI changes.
- **Backend Engineering with Node.js and Express.js:** The backend applied core Web Engineering principles through RESTful API development, middleware logic, and secure request handling. Validation mechanisms such as supervisor capacity checks, duplicate request prevention, and group size enforcement were implemented in Express middleware. The backend manages asynchronous operations, structured routing, and role-based access, ensuring the system behaves coherently and efficiently under real academic workflow scenarios.

1.2.3 Database Systems

This module guided the organization, structuring, and management of the portal's cloud-hosted data.

- **Data Modelling and Structure Design:** The MongoDB database schema was designed using Database Systems concepts such as normalization strategies, indexing, efficient retrieval methods, and structured relationships between Users, Groups, Notifications, and Submissions. Logical modelling ensured that operations like fetching supervisor availability, verifying group capacity, or checking submission history were optimized for performance and consistency.
- **Cloud-Based NoSQL Operations:** Knowledge of CRUD operations, atomic updates, and secure data access informed how real-time academic constraints were enforced through database logic. MongoDB Atlas provided scalable, fault-tolerant data handling while maintaining data integrity for sensitive academic information. These concepts were essential for managing dynamic and high-frequency operations such as request validations and notification updates.

1.2.4 Human-Computer Interaction (HCI)

This module influenced the usability, interface clarity, and overall user experience of the system.

- **User-Centered Interface Design:** HCI principles guided the creation of dashboards tailored to each user role, ensuring minimal cognitive load and intuitive navigation. Visual hierarchy, colour cues, spacing, and interaction flow were designed to highlight important functions such as supervisor availability, group requests, notifications, and submissions. The interface is streamlined so that each user only sees controls relevant to their responsibilities.
- **Interaction Flow and Accessibility:** Concepts such as error prevention, timely feedback, and consistent design patterns were applied to features like real-time notifications, confirmation messages, and input validations (e.g., "Group Full" or "Request Already Sent"). These help users' complete tasks smoothly while reducing confusion and preventing invalid actions. The interface remains accessible to non-technical users, reflecting HCI's emphasis on usability and accessibility.

1.2.5 Requirement Engineering

This module shaped how system needs were gathered, documented, and translated into functional features.

- **Requirement Elicitation and Analysis:** Requirement Engineering techniques such as interviews, workflow observations, and scenario analysis informed how functional requirements were extracted from real academic processes. Needs such as group creation, supervisor limit checks, real-time constraints, and administrative controls were documented to ensure the system addressed genuine institutional challenges. These requirements formed the basis of the entire system logic.
- **Documentation and Specification:** Detailed requirement documents, including use cases, process descriptions, and system constraints, were prepared using Requirement Engineering methods. These specifications acted as the blueprint for developers, ensuring features were implemented correctly and consistently. Clear requirement definitions prevented miscommunication and ensured alignment between frontend, backend, and database components.

1.2.6 Software Testing

This module ensured the reliability, correctness, and robustness of the FYP Portal.

- **Validation of Functional Logic:** Software Testing techniques were applied to verify critical backend behaviours such as capacity checks, authentication workflows, and prevention of invalid actions. Unit tests and logical validations ensured that supervisor limits, group sizes, and duplicate requests were enforced accurately. These tests guaranteed that the system responded correctly to both normal and edge-case scenarios.
- **Integration & System Testing:** Frontend and backend integration was tested using scenario-based workflows to validate real student-supervisor interactions. Regression testing ensured new updates did not break existing functionality, while usability testing evaluated whether non-technical users could navigate the system easily. This process improved stability, performance, and reliability across all modules of the portal.

1.2.7 Information Security

This module shaped the secure handling of sensitive academic data and user permissions.

- **Secure Authentication and Role-Based Access:** Information Security concepts guided the implementation of encrypted password storage, secure authentication flows, and strict role-based access. Only admin-approved users can log in, reducing unauthorized access risks. Sensitive actions, such as accepting group requests or modifying limits, are protected through controlled permissions.
- **Data Protection & Integrity:** Security principles such as confidentiality, integrity, and availability (CIA triad) influenced how user sessions, academic submissions, and group data were safeguarded. Input validation, secured API communication, and controlled database access ensure protection against misuse or data corruption. These measures maintain trust, reliability, and the academic integrity of the portal.

1.3 PROJECT BACKGROUND

The management of Final Year Projects has historically been one of the most challenging responsibilities within academic departments, often constrained by manual practices, scattered communication channels, and inconsistent documentation methods. Traditionally, students form groups through informal discussions, reach out to supervisors individually, submit documents through email or messaging apps, and receive feedback without any structured record-keeping. This fragmented workflow leads to frequent miscommunication, unclear supervision responsibilities, difficulty tracking progress, and loss of important project-related files. As universities continue to grow in size and academic complexity, the limitations of these manual systems become more apparent, creating a strong need for a centralized and reliable digital solution.

The origins of the FYP Portal lie in understanding these long-standing issues faced by students, supervisors, HODs, and administrative staff. Students often struggle to determine which supervisors are available, what rules govern group formation, or how to communicate progress properly. Supervisors, on the other hand, face difficulty managing multiple groups simultaneously when student requests, submissions, and messages arrive through different channels. Without a unified system, HODs lack a comprehensive overview of departmental FYP activities, making it challenging to ensure fair supervisor load distribution or track the progress of all registered groups. These gaps highlighted the need for a standardized and automated academic management platform.

The FYP Portal was designed to address this gap by offering a dedicated website that brings all FYP-related operations into one coherent digital ecosystem. The portal organizes work through four distinct user roles, Student, Supervisor, HOD, and Admin, each with separate dashboards and functionalities that replicate real academic workflows. Students can register with university-provided credentials, form groups, join existing groups using group IDs, select supervisors based on availability, and submit documents directly through the portal. Supervisors receive group requests instantly, accept or reject students, monitor progress updates, and communicate through an integrated messaging system. This structured workflow eliminates ambiguity and establishes clear accountability across all participants.

At a departmental level, the HOD gains complete visibility into all registered groups, their supervisors, and their progress. This centralized oversight enables the HOD to identify delays

early, distribute workload evenly, and broadcast important announcements directly through the portal. The administrative panel plays an equally crucial role, allowing the admin to create or remove student and supervisor accounts, generate login credentials, manage supervisor limits, set group size restrictions, and ensure all academic rules are enforced in real time. Whenever limits are reached, such as a supervisor having three groups or a group being full, the system automatically prevents further requests and informs the user instantly.

The technical foundation of the FYP Portal further strengthens its role in modern academic management. Developed using React.js for the frontend, Node.js with Express.js for the backend, and MongoDB Atlas as the cloud database, the system ensures scalability, responsiveness, and secure data handling. These technologies were chosen to support dynamic operations such as real-time notifications, instant validation checks, and continuous interactions between different user roles. By adopting a web-based architecture, the portal remains accessible from any device without requiring installation, making it highly practical for everyday academic use.

In summary, the FYP Portal was developed as a direct response to the inefficiencies in traditional FYP supervision methods. Its purpose is to provide transparency, consistency, and structure to the entire project lifecycle. Through automation, digital documentation, systematic supervision, and centralized oversight, the portal transforms the way FYPs are managed within the department. It not only simplifies academic workflows but also enhances fairness, reduces workload on faculty, and improves the overall experience for students embarking on their final-year projects.

1.4 Literature Review

The management of academic projects has been widely discussed in educational technology research, with many studies highlighting the need for structured, digital, and automated supervision systems. Traditional methods, relying on email threads, shared folders, and informal communication, are frequently criticized for their inefficiencies and lack of traceability.[5] Modern academic institutions increasingly require platforms that support transparent supervision, centralized document management, and automated enforcement of academic policies.

This literature review examines existing tools, platforms, and research findings that relate to the development of the FYP Portal. It evaluates their strengths and limitations to establish the academic justification for creating a dedicated web-based Final Year Project management system.

1.4.1 Existing Academic Management Systems

Research shows that various Learning Management Systems (LMS) such as Moodle, Google Classroom, Microsoft Teams, Edmodo, and Canvas have been widely adopted for course delivery, assignment submission, and communication. These platforms provide structured learning environments, offering features such as grading tools, discussion forums, and resource sharing. However, their focus is primarily on classroom-based learning rather than project-based supervision.

These systems lack dedicated mechanisms for supervisor assignment, multi-role workflows, group capacity limits, academic constraints, or real-time validation required for FYP management. Even though Google Classroom allows teachers to assign and grade work, it does

not support university-specific rules such as maximum three groups per supervisor, group ID–based joining, *or* automated request approval workflows.

Similarly, Microsoft Teams enables communication and document sharing, but supervisory tasks become fragmented across chats, folders, and channels. There is no structured FYP lifecycle built into the platform, making long-term tracking of progress difficult. Research comparing LMS platforms consistently highlights the absence of academic project workflow features, demonstrating the need for specialized systems that address FYP-specific requirements.

1.4.2 Project Management Tools and Their Limitations

Commercial project management tools such as Trello, Asana, Jira, Basecamp, and Notion have been explored in academic research for their potential use in student projects. These platforms offer task boards, progress tracking, file attachments, and team collaboration. While they are effective for general project coordination, studies show they are often too complex, require customization, and do not naturally align with academic rules.

Tools like Salesforce or Jira demand technical knowledge and configuration expertise that students and supervisors may not possess. Furthermore, they lack built-in university governance features such as:

- Role separation between student, supervisor, and HOD
- Auto-validation rules (group size, supervisor availability)
- Academically structured submission cycles
- Department-level visibility

These tools are powerful but generic. Academic literature emphasizes that such platforms cannot enforce institutional policies or manage supervisory hierarchies without significant modifications. This supports the argument that a custom-built system, tailored to academic workflows, is more suitable for FYP management.

1.4.3 Studies on Academic Supervision and Digital Transformation

Multiple research papers address the challenges faced in monitoring academic project progress. Scholars highlight issues such as inconsistent communication, unclear expectations, documentation loss, and unfair supervisor allocation. Traditional supervision methods rely heavily on personal initiative, which creates disparities among students and increases the workload for faculty members.

Studies on digital supervision systems indicate that centralized platforms improve communication clarity, reduce errors, and increase student satisfaction. These systems encourage timely interactions between supervisors and students, provide structured timelines, and enhance transparency through documented communication histories. However, existing research also reveals that many available systems are not web-based, lack scalability, or do not integrate modern technologies like real-time validation or cloud-based storage.

Universities adopting digital solutions often report improved coordination but still require tools that incorporate role-based restrictions, academic constraints, and automated decision-making, which the FYP Portal addresses directly.

1.4.4 Gap in Existing Solutions and Need for a Dedicated FYP Portal

From the literature review, a significant gap emerges: there is no widely adopted platform designed specifically for the complete lifecycle of Final Year Project supervision. Existing LMS platforms focus on course delivery, not multi-role project supervision. Generic project management tools focus on corporate workflows, not academic rules. Research prototypes lack scalability, cloud integration, or real-time enforcement features.

The FYP Portal fills this gap by providing a platform that:

- Supports four clearly defined academic roles
- Enforces real-time supervisor and group limits
- Structures submission, communication, and feedback workflows
- Provides centralized visibility for HODs and admins
- Offers automated notifications and validation
- Ensures secure handling of academic data

Existing literature strongly supports the idea that academic institutions benefit from platforms designed specifically for FYP management. The FYP Portal aligns with these findings by offering a system tailored to the challenges and requirements identified across multiple studies.

1.5 Analysis From Literature Review

The literature reveals that while many digital tools exist for communication, assignment submission, and project coordination, none of them adequately satisfy the unique requirements of Final Year Project management within academic institutions. Systems such as Google Classroom, Moodle, Microsoft Teams, and other LMS platforms are designed primarily for general coursework activities rather than structured project supervision.[6] These platforms fail to incorporate academic restrictions such as supervisor capacity limits, controlled group formation, or automated validation mechanisms. This gap highlights the fundamental need for a customized platform capable of addressing the specific challenges and workflows associated with FYP supervision.

Further analysis of project management tools like Trello, Asana, Jira, and Basecamp shows that although they offer advanced task tracking and collaboration features, they are not suitable for academic governance. Their complexity, lack of supervisor-student hierarchy, absence of academic policy enforcement, and inability to manage institution-specific rules make them unsuitable for FYP environments. These tools cater to corporate task management and team-based development rather than academic mentorship structures. This mismatch reinforces the need for a system purpose-built to handle the structured, multi-role interactions found in academic departments.

Research studies also emphasize recurring supervision issues such as inconsistent feedback, communication delays, undocumented progress, and non-standardized evaluation criteria. The need for transparency, centralized documentation, and structured communication between supervisors and students is a recurring theme across academic literature. These findings align strongly with the core objectives of the FYP Portal, which seeks to eliminate ambiguity through automated request handling, real-time notifications, defined user roles, and centralized submission records. The system directly responds to research-documented challenges, offering structured workflows that improve clarity and accountability.

Finally, the literature shows that administrative oversight in academic projects is often limited by the lack of real-time visibility into student progress and supervisor workload distribution. Without a centralized system, HODs struggle to monitor departmental performance accurately, and admins must manually enforce academic rules. This creates inefficiencies and increases the likelihood of errors. The FYP Portal resolves these issues by providing a unified digital platform that automates administrative policies, enforces supervision limits, and offers real-time dashboards for HODs and admins. This analysis confirms that the FYP Portal is not only aligned with the needs identified in academic research but also fills significant gaps left by existing systems.

1.6 Methodology and Software Lifecycle

The FYP Portal project adopted an Iterative and Incremental Software Development Life Cycle (SDLC) model paired with flexible development practices inspired by Agile principles. These approaches were selected due to their suitability for handling complex, multi-role academic workflows such as student group formation, supervisor management, real-time notifications, and dynamic constraint validation. Because a project like FYP Portal evolves through multiple layers of academic requirements and policy-driven rules, the development model needed to support continuous refinement, parallel development, and adaptability. The iterative approach allowed the development team to gradually implement system modules, including authentication, group logic, supervisor request handling, submission management, and admin controls, while continuously integrating feedback and improving functionality.

Inspired by Agile methodology, development work was divided into cycles, each focusing on building and stabilizing a specific module of the system. For example, one iteration focused on implementing student registration and login flows, another on supervisor capacity logic, while later cycles refined HOD dashboards, messaging features, and admin-level configuration tools. This structure allowed developers to test each module independently and ensure early identification of issues such as validation errors, inconsistent workflows, UI misalignment, or backend logic conflicts. Regular reviews and repeated UI-backend integrations provided clarity and helped maintain alignment with academic expectations for FYP supervision.

The iterative model also supported dynamic changes to academic rules that emerged during development. For instance, adjustments to the group size limit, refinements in supervisor availability logic, or improved handling of duplicate requests were implemented smoothly without requiring a redesign of the entire system. By delivering the system in increments, the team ensured that each major component was fully functional before progressing to the next layer. This approach led to a stable, scalable, and highly structured academic portal capable of supporting real-time updates, secure data handling, and extensive multi-user interactions.

Ultimately, the combination of iterative development and Agile-inspired flexibility enabled the team to produce a robust, user-centered system within the academic timeline. The chosen methodology ensured that the portal remained adaptable, aligned with departmental policies, and ready for future enhancements such as advanced analytics, bulk student uploads, and expanded supervisor management tools.

1.6.1 Rationale Behind Selected Iterative Development Model

The Iterative Development Model was selected for the FYP Portal due to its strong alignment with the system's academic and technical requirements. The project is centered around four distinct roles, Student, Supervisor, HOD, and Admin, each requiring unique interfaces and

workflows, making a linear model like Waterfall unsuitable. Iterative development allowed the team to refine these complex features in cycles, ensuring each role-based module was stable, functional, and integrated correctly before expanding to the next.

Key drivers for selecting this model include:

- **Flexibility to Accommodate Changing Academic Rules:** The FYP Portal incorporates real-time constraints such as supervisor group limits, student group size restrictions, and request validation logic. These requirements evolved as the system was tested and reviewed. The iterative model provided the necessary flexibility to adjust constraints, modify validation flows, and redesign interactions without disrupting the entire development process.
- **Incremental Implementation of Multi-Role Workflows:** Because each module required significant logic, such as handling supervisor approvals, managing submissions, or enforcing admin-set limits, implementing these features incrementally ensured stability and reliability. Each iteration produced a working component, such as the student dashboard or supervisor request engine, which was tested thoroughly before moving to the next development cycle.
- **Continuous Feedback and Real-World Scenario Validation:** Throughout development, workflows were evaluated using actual academic scenarios (students joining full groups, supervisors exceeding capacity, admin updating constraints, etc.). This continuous feedback loop helped refine UX behaviour, error messages, backend logic, and dashboard clarity, ensuring the portal behaved exactly as intended in a real FYP environment.

1.6.2 Rationale Behind Selected Methodology

The methodology selected for the FYP Portal was driven by the need for both reliability and adaptability, qualities essential for systems interacting with academic institutions. The portal required precise enforcement of rules, secure authentication, structured communication, and clear role-based interfaces, all of which benefit from a controlled, iterative development workflow. The integration of object-oriented thinking further enhanced the development process by enabling modularization of backend controllers, reusable frontend components, and clearly structured database collections.

The iterative model supported incremental feature deployment, which was necessary because academic processes must be validated step-by-step. For example, supervisor approval flows, notification handling, and document submission logic were developed, tested, and refined in successive cycles. This prevented large-scale errors, improved system usability, and ensured alignment with departmental policies. The approach also enhanced scalability, allowing the portal to accommodate future features such as bulk data uploads, supervisor workload analytics, or improved messaging capabilities without requiring major architectural changes.

By combining structured analysis, object-oriented design, and iterative development, the methodology ensured that the FYP Portal remained robust, user-friendly, and technically sound. This approach ultimately delivered a high-quality academic management system suited for real-world deployment within university environments.

Chapter 2 Problem Definition

2 Problem Definition

The successful execution of Final Year Projects (FYPs) depends heavily on effective coordination between students, supervisors, departmental administrators, and the Head of Department. Yet, in many academic institutions, FYP management still relies on manual processes such as email exchanges, informal messaging, handwritten notes, or verbal communication. These fragmented methods result in delays, miscommunication, lack of standardization, and difficulty in tracking academic progress. Without a centralized system, students face confusion during group formation, supervisors struggle to monitor multiple teams, and administrators lack the tools to enforce institutional policies fairly.

The absence of an integrated digital platform also leads to inefficiencies in supervision workflows, document submission, and notification delivery. Important academic rules, such as supervisor capacity limits or maximum group size, are often enforced inconsistently, simply because there is no automated mechanism to validate them. As universities expand and project requirements become more demanding, the need for a structured, web-based FYP management system becomes increasingly critical. The FYP Portal addresses this gap by providing a unified, automated, and transparent environment for managing the entire FYP lifecycle.

2.1 Problem Statement

In many academic institutions, the Final Year Project (FYP) process is still managed through traditional and fragmented methods such as emails, WhatsApp groups, physical submissions, spreadsheets, and informal verbal communication. These manual workflows often lead to confusion among students, who struggle to identify available supervisors, understand departmental rules, form or join groups correctly, and keep track of submission deadlines. Important announcements and updates are frequently missed, documents are scattered across multiple platforms, and students lack a clear, unified view of their project progress. As a result, the overall efficiency and effectiveness of the FYP process are significantly reduced.

Supervisors also encounter substantial difficulties when managing multiple student groups without a centralized digital system. Requests from students are often received through unstructured channels, making it difficult to validate eligibility, track responses, or maintain consistent communication. Monitoring project progress, reviewing submissions, and providing timely feedback becomes challenging due to irregular document handling and the absence of a standardized workflow. Additionally, enforcing academic constraints such as maximum supervision limits, group size restrictions, and prevention of duplicate supervisor requests is nearly impossible through manual processes.

The Head of Department (HOD) and administrative staff face similar challenges due to the lack of real-time visibility into ongoing FYP activities. Without a centralized platform, it is difficult to monitor supervisor workloads, ensure equitable distribution of projects, or verify compliance with departmental policies. Administrative tasks such as managing student and supervisor records, enforcing institutional rules, and responding to academic queries require excessive manual effort and are prone to errors, inconsistencies, and delays. This lack of oversight negatively impacts academic governance and decision-making.

The absence of an integrated FYP management system result in miscommunication, inefficiencies, academic inconsistencies, and unnecessary workload for all stakeholders involved. Therefore, there is a clear need for a dedicated web-based platform that centralizes

FYP workflows, automates academic rules, and provides structured, role-based dashboards for students, supervisors, HODs, and administrators. The proposed FYP Portal addresses these issues by offering real-time validation, transparent communication channels, secure data handling, and a unified environment tailored specifically for academic project supervision. By digitizing and organizing the entire FYP lifecycle, the portal ensures an efficient, fair, and well-coordinated academic process.

2.2 Deliverables and Development Requirements

The FYP Portal consists of several key deliverables designed to digitalize, centralize, and streamline the entire Final Year Project lifecycle. These deliverables aim to eliminate manual and fragmented processes by providing a unified web-based platform where all academic activities related to FYP management are handled systematically. Through structured workflows, the portal ensures that students can form groups, request supervisors, submit documents, and track progress in a clear and organized manner, reducing confusion and administrative delays.

Additionally, the portal supports supervisors, HODs, and administrators by offering role-based dashboards, automated rule enforcement, and real-time notifications. Academic constraints such as group size limits, supervisor capacity, and submission validation are enforced automatically, ensuring fairness and consistency across the department. Together, these deliverables create a reliable and transparent academic environment that improves coordination, enhances supervision quality, and supports effective decision-making throughout the FYP process.

2.2.1 Fully Functional Web-Based FYP Management Portal

The primary deliverable of the FYP Portal is a complete web-based system that provides role-specific dashboards for Students, Supervisors, the Head of Department (HOD), and Administrators. Each user role is presented with a tailored interface that reflects its responsibilities within the FYP lifecycle. Students can create or join project groups, request supervisors, upload required documents, and continuously track their project progress. This structured approach ensures that all academic activities follow predefined workflows, minimizing errors and miscommunication.

Supervisors are provided with dedicated tools to review student requests, monitor assigned groups, evaluate submissions, and communicate feedback through the portal. The HOD gains a centralized overview of all supervisors and registered groups, enabling effective oversight and policy enforcement, while administrators manage user accounts and academic constraints such as group size and supervision limits. The portal is designed to be responsive, user-friendly, and accessible through standard web browsers, ensuring ease of use across different devices without requiring mobile application support.

2.2.2 Supervisor and Group Management Features

A dedicated module manages all workflows involving group formation and supervisor selection. The system enforces academic rules such as supervisor capacity limits, maximum group size, and prevention of duplicate requests. Supervisors can accept or reject incoming student requests, view group details, and monitor ongoing submissions within a structured interface. These automated controls eliminate errors and ensure fairness across the departmentA dedicated module within the FYP Portal is responsible for managing all workflows related to group formation and supervisor selection. This module automates critical

academic rules, including maximum group size, supervisor supervision limits, and restrictions on duplicate or conflicting requests. When a student attempts to join a group or request a supervisor, the system performs real-time validation to ensure compliance with departmental policies before allowing the action. This structured validation removes ambiguity, prevents invalid operations, and guides students through a clear and fair selection process.

Supervisors interact with this module through a centralized dashboard where they can review incoming requests, accept or reject group associations, and view complete group details once registered. The interface also allows supervisors to monitor ongoing submissions and project progress for each assigned group without relying on external communication tools. By automating supervision constraints and request handling, the portal ensures balanced workload distribution, reduces administrative effort, and promotes transparency and fairness across the department.

2.2.3 HOD Oversight and Announcement Tools

The FYP Portal includes a dedicated HOD dashboard that provides comprehensive visibility into all registered groups and supervisors within the department. Through this dashboard, the HOD can view which groups are assigned to which supervisors, monitor overall project distribution, and track progress at a departmental level. This centralized overview enables the identification of workload imbalances, inactive groups, or supervision gaps, supporting informed academic and administrative decision-making.

In addition to monitoring activities, the HOD can use the portal to publish announcements, guidelines, and official notices that are instantly delivered to all students and supervisors. These notifications appear directly on user dashboards, ensuring that critical academic information such as deadlines, templates, or policy updates is communicated consistently and without delay. This structured communication mechanism eliminates reliance on informal channels and strengthens transparency and coordination across the department.

2.2.4 Admin Panel for User and Policy Management

The admin panel serves as the central control unit of the FYP Portal, enabling administrators to create, update, and manage student and supervisor accounts efficiently. Through this interface, the admin can generate login credentials, remove inactive users, and maintain an accurate academic database. This centralized account management ensures that only authorized users can access the system and that all participants are correctly mapped to their academic roles.

In addition to user management, the admin panel allows configuration of critical academic rules such as maximum group size and the number of groups a supervisor can oversee. Any changes made by the admin are enforced in real time across the portal, preventing invalid actions such as overfilled groups or excess supervision assignments. This ensures consistency, transparency, and strict adherence to departmental policies throughout the FYP lifecycle.

2.2.5 Backend Validation and Notification Engine

The FYP Portal incorporates a dedicated backend validation engine that governs all user actions based on predefined departmental rules. Whenever a student attempts to create or join a group, send a supervisor request, or submit project material, the system automatically verifies eligibility conditions such as group capacity, supervisor availability, and duplicate request

prevention. This centralized validation mechanism ensures that only valid actions are processed, eliminating manual checks and reducing academic inconsistencies.

Working in parallel with the validation layer, the notification engine generates real-time alerts for critical system events, including supervisor approvals or rejections, new student requests, document submissions, and official announcements. These notifications are delivered instantly to relevant dashboards, ensuring timely awareness and response from all stakeholders. Together, the validation and notification components enable smooth communication, enforce correct system behaviour, and support a transparent and well-coordinated FYP management process.

2.3 Development Requirements

The development requirements for the FYP Portal outline the technical, environmental, and operational foundations necessary to implement a fully functional, secure, and scalable web-based academic management system. These requirements ensure that the platform can support multi-role interactions, automated validations, real-time notifications, and maintain overall performance during active academic cycles. The following subsections describe the essential components needed for successful development and deployment of the system.

2.3.1 Technical Stack

The FYP Portal is built using a modern and flexible technical stack designed to support dynamic operations and multi-user workflows.

- **Frontend:** Developed using React.js, which enables component-based UI design, efficient state management, and responsive dashboards for Students, Supervisors, HOD, and Admin.
- **Backend:** Implemented using Node.js and Express.js to handle API routing, business logic, authentication, and system-wide validations.
- **Database:** MongoDB Atlas serves as the primary database, offering scalable cloud-hosted document storage suitable for user profiles, group data, submissions, notifications, and system rules.

This technical stack was selected for its adaptability, performance, and ability to handle real-time updates in academic workflows.

2.3.2 Environment Setup

A suitable development environment is required to support efficient coding, testing, and deployment.

- **Node.js & npm:** Essential for backend development, API testing, and installing required packages.
- **React Development Environment:** Created using tools like Vite or Create React App, enabling modular UI development with live updates.
- **MongoDB Atlas Configuration:** A cloud database environment must be configured with secure access rules, collections, and indexes for optimized performance.
- **Testing Tools & Local Servers:** Postman, browser developer tools, and Node-based local servers are used to simulate backend calls and validate functionality before deployment.

2.3.3 Security Requirements

Because the portal handles sensitive academic data and user credentials, strict security requirements must be applied.

- **Password Encryption:** All passwords must be securely hashed before storage to prevent unauthorized access.
- **Role-Based Access Control:** Each user role, Student, Supervisor, HOD, Admin, must only access features have assigned to their permissions.
- **Validation Rules:** Inputs must be sanitized to prevent misuse, unauthorized actions, or bypassing academic constraints.
- **Database Security:** MongoDB Atlas must enforce IP restrictions, encrypted connections, and secure user authentication.

2.3.4 Performance And Scalability Requirements

To ensure reliability during active academic periods, the system must maintain strong performance under varying workloads.

- **Efficient API Handling:** The backend should process multiple simultaneous requests without delays, especially for group management and supervisor approvals
- **Optimized Database Operations:** Queries must be indexed and structured for fast retrieval of student records, requests, and notifications.
- **Scalability:** The system should support an increasing number of students, supervisors, submissions, and notifications without performance degradation.
- **Responsive Frontend:** Dashboards must load quickly and update in real time to reflect new requests, approvals, and announcements.

Chapter 3 Requirement Analysis

3 Requirement Analysis

This chapter translates the project goals into precise system requirements. It describes how the FYP Portal must behave (functional requirements), the quality attributes it must satisfy (non-functional requirements), and the actor-level interactions captured by use cases. The analysis below is based on stakeholder needs (students, supervisors, HOD, admins), the project scope, and the academic constraints (supervisor limits, group sizes, approval workflows).

3.1 Use Case Diagram

The use-case diagram visualizes how different actors interact with the FYP Portal to achieve key goals (register/login, form/join groups, request supervision, submit documents, monitor progress, and administer the system).

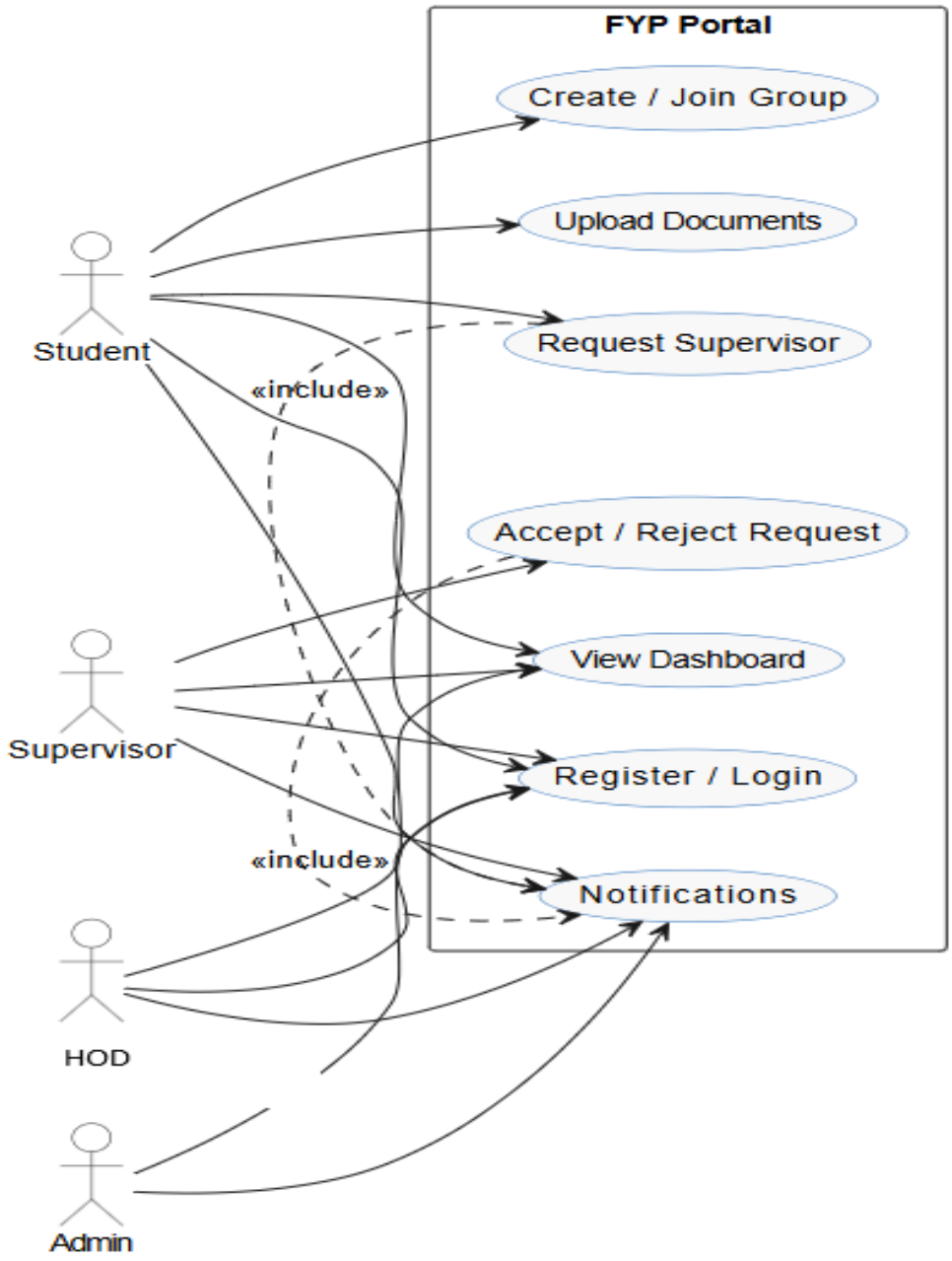

Figure 1: Use Case Diagram

3.2 Detailed Use Case

Below are detailed use-case descriptions tailored to the FYP Portal.

Table 1: Use Case 1

Use Case id	01
Use case name	Login / Register
Actors	Student, Supervisor, HOD, Admin
Description	Allows an authorized user to log in using university-provided credentials (or admin-created account) and establishes a session. New accounts (if allowed) are verified and activated by Admin.
Trigger	User attempts to access the portal.
Pre-condition	Account exists (created by Admin) and server reachable.
Post-condition	User authenticated and redirected to the proper dashboard (role-based).
Normal flow	User opens login page and enters credentials. System validates credentials and role. System creates session and routes user to role dashboard.
Alternative flow	Forgot password → System prompts email → email with reset link.
Exceptions	Invalid credentials, account suspended, network error.
Business rules	Only admin-created accounts (or verified accounts) may access the portal. Passwords must meet complexity requirements.

Table 2: Use Case 2

Use Case id	02
Use case name	Create Group
Actors	Student
Description	Student creates a new project group and receives a generated Group ID for others to join.
Trigger	Student decides to start a new group.
Pre-condition	Student is authenticated and not already in a full group.
Post-condition	Group created, student becomes group owner, Group ID generated.

Normal flow	Student fills group details → system validates → group saved and Group ID shown.
Alternative flow	Missing required info → system prompts to complete fields.
Exceptions	Student already in a group that prevents creating (policy), DB error.
Business rules	A group cannot exceed the system-defined maximum size.

Table 3: Use Case 3

Use Case id	03
Use case name	Join Group
Actors	Student
Description	Student enters an existing Group ID to request joining that group.
Trigger	Student provides Group ID and requests to join.
Pre-condition	Group exists and not full.
Post-condition	Join request sent to group owner/supervisor (if applicable) or student added if auto-join allowed.
Normal flow	Student inputs Group ID → system checks capacity → request created and appropriate notifications sent.
Alternative flow	Group full → system notifies student.
Exceptions	Invalid Group ID, network error.
Business rules	Duplicate join requests by the same student are rejected.

Table 4: Use Case 4

Use Case id	04
Use case name	Request Supervisor
Actors	Student
Description	Student selects a supervisor from available pool and sends a supervision request.
Trigger	Student clicks “Request Supervisor” on the chosen supervisor profile.

Pre-condition	Student in a group and supervisor has remaining capacity.
Post-condition	Request recorded and notification sent to the supervisor.
Normal flow	System checks supervisor load → sends request → supervisor notified.
Alternative flow	Supervisor limit full → system blocks request and informs student.
Exceptions	Supervisor record not found, concurrent acceptance issue.
Business rules	Supervisors cannot supervise more than N groups (admin-configured).

Table 5: Use Case 5

Use Case id	05
Use case name	Supervisor Response (Accept/Reject)
Actors	Supervisor
Description	Supervisor reviews pending requests and accepts or rejects. Acceptance assigns the group under their supervision.
Trigger	Supervisor opens pending requests list.
Pre-condition	Requests exist and supervisor not exceeding capacity.
Post-condition	Request status updated; student(s) notified.
Normal flow	Supervisor reviews request → accepts → system updates group-supervisor relation and adjusts capacity.
Alternative flow	Supervisor rejects → reason optionally provided → student notified.
Exceptions	Race conditions where multiple supervisors accept at once (system must enforce single assignment).
Business rules	Supervisor acceptance reduces available slots by one.

Table 6: Use Case 6

Use Case id	06
Use case name	Upload Documents / Share Progress
Actors	Student, Supervisor

Description	Students upload project files (reports, code, presentations); supervisors can upload feedback files.
Trigger	User clicks “Upload” on submissions area.
Pre-condition	User authenticated and group membership verified.
Post-condition	File stored (metadata saved) and visible to authorized roles.
Normal flow	User selects file → system validates type/size → stores file → creates metadata record → notifies supervisor / group members.
Alternative flow	File too large → system prompts compress / split.
Exceptions	Unsupported file type, storage limit exceeded.
Business rules	Only accepted file types allowed; submission history retained.

Table 7: Use Case 7

Use Case id	07
Use case name	HOD Announcements / Oversight
Actors	HOD
Description	HOD views all groups, supervisor assignments, and publishes department-wide announcements or templates.
Trigger	HOD accesses the HOD dashboard.
Pre-condition	HOD authenticated.
Post-condition	Announcement’s broadcast; data viewable by admins, supervisors, students as defined.
Normal flow	HOD composes announcement → system validates → broadcasts via notifications.
Alternative flow	Announcement scheduled for future → system queues it.
Exceptions	Authorization failure.
Business rules	Announcements must be verifiable and may be restricted to certain audiences.

Table 8: Use Case 8

Use Case id	08

Use case name	Admin Manage Users & Limits
Actors	Admin
Description	Admin creates/removes user accounts, sets supervisor capacity, sets group size limits, and manages templates.
Trigger	Admin logs into admin panel and modifies settings.
Pre-condition	Admin authenticated and authorized.
Post-condition	System rules updated; users notified if relevant.
Normal flow	Admin updates limits → system persists changes → system enforces new rules immediately.
Alternative flow	Admin schedules changes for next semester.
Exceptions	Unauthorized attempt, DB errors.
Business rules	Admin actions are audited.

3.3 Functional Requirements

The functional requirements of the FYP Portal define the core operations that the system must support to facilitate an efficient and structured final-year project process. These include secure user authentication, student group creation and joining, supervisor selection and request handling, document uploading, progress tracking, and real-time notification delivery. Each function is designed to guide users through clearly defined workflows, reducing ambiguity and ensuring consistent academic procedures.

In addition, the system must support role-specific actions to maintain coordination across all stakeholders. Supervisors are required to review student requests, monitor project progress, and provide feedback through the portal, while the HOD must have access to a centralized overview of all groups and the ability to distribute announcements. Administrative functions include user account management and configuration of system constraints such as group size and supervisor capacity. Collectively, these functional requirements ensure that the portal operates as a unified, reliable, and academically compliant FYP management system.

Table 9: Functional Requirements

Requirement ID	Requirement Title	Source	Rationale	Business Rule (if any)	Dependencies	Priority
FR-01	User Login / Authentication	Use Cases	Secure access to role-based dashboards	Only admin-created/verified accounts allowed	Authentication service	High
FR-02	Create / Join Group	Use Cases	Allow students to form and	Group cannot exceed max size	Group service, DB	High

			join project groups			
FR-03	Request Supervisor	Use Case	Enable students to request supervisors	Supervisor capacity must not be exceeded	Notification & validation engine	High
FR-04	Supervisor Accept/Reject	Use Case	Allow supervisors to manage their groups	Supervisor limit enforced	Validation engine	High
FR-05	Upload Documents & Submission History	Use Case	Enable document submission and history tracking	File types & size rules apply	Storage service	High
FR-06	Notifications & Alerts	Use Cases	Inform users of request status and announcements	Real-time notifications for key events	Notification service	High
FR-07	HOD Dashboard & Announcements	Use Case	Departmental overview and communication	Announcements broadcast to intended audience	Notification service	Medium
FR-08	Admin Management & Policy Settings	Use Case	Administer accounts and rules	Admin actions are audited	Admin panel	High
FR-09	Validation & Limits Engine	Cross cutting	Automatically enforce academic policies	Enforce group size and supervisor capacity	Backend validation	Critical

3.4 Non-Functional Requirements

The non-functional requirements of the FYP Portal define the quality attributes that ensure the system operates in a reliable, secure, and user-friendly manner. These requirements focus on usability by providing an intuitive and easy-to-navigate interface for all user roles, ensuring that students, supervisors, and administrators can perform their tasks without unnecessary complexity. Performance requirements ensure that user actions such as login, dashboard loading, and submission uploads are processed efficiently with minimal delay.

In addition, the non-functional requirements address system security, reliability, and scalability. Security measures ensure that user data and academic records are protected through proper authentication and role-based access control. Reliability requirements ensure stable

system availability and consistent operation throughout the academic cycle, while scalability allows the portal to accommodate an increasing number of users, groups, and future feature enhancements. Together, these attributes ensure that the FYP Portal delivers a dependable, secure, and high-quality academic management experience.

3.4.1 Usability Requirements

These requirements define how easily users can interact with the FYP Portal and navigate its features with minimal effort.

Table 10: Usability Requirements

ID	Requirement Description
USE-01	The UI must be user-friendly; common tasks (create/join group, upload docs) should complete in ≤ 3 clicks.
USE-02	Dashboards must present role-specific information clearly (Students, Supervisors, HOD, Admin).
USE-03	The interface must be accessible (keyboard navigable, proper contrasts) and responsive on desktop browsers.

3.4.2 Performance Requirements`

These requirements specify the expected speed, responsiveness, and efficiency of the system during normal and peak usage.

Table 11: Performance Requirements

ID	Requirement Description
PER-01	System should support at least 2,000 concurrent active users (configurable for scale).
PER-02	Typical dashboard page load must be ≤ 2 seconds on broadband connections.
PER-03	Validation responses (e.g., checking supervisor capacity) must respond within 500ms.

3.4.3 Security Requirements

These requirements ensure that all user data, actions, and system resources are protected from unauthorized access or misuse.

Table 12: Security Requirements

ID	Requirement Description

SEC-01	All communications must use HTTPS/TLS.
SEC-02	Passwords must be stored hashed (bcrypt/argon2) and never logged in plaintext.
SEC-03	Role-Based Access Control: only permitted roles can perform privileged functions (admin, HOD, supervisor).
SEC-04	Inputs must be validated/sanitized to prevent injection (NoSQL/JS injection).

3.4.4 Reliability Requirements

These requirements describe the system's ability to operate consistently without failures and maintain dependable service availability

Table 13: Reliability Requirements

ID	Requirement Description
REL-01	System uptime should target 99.9% (excluding scheduled maintenance).
REL-02	The system should support automatic recovery on transient failures (retry policies / queued jobs).
REL-03	Backups of critical data (users, groups, submissions) must be taken daily and retained per policy.

3.4.5 Scalability Requirements

These requirements outline how well the system can accommodate increased users, data, and workloads as the portal grows over time.

Table 14: Scalability Requirements

ID	Requirement Description
SCA-01	Application must be horizontally scalable (stateless web servers, DB scaling).
SCA-02	Codebase must be modular to allow independent scaling of API and notification services.
SCA-03	Storage must support increasing submission volume (use cloud storage with lifecycle rules).

3.4.6 Maintainability Requirements

These requirements define how easily the FYP Portal can be updated, corrected, and enhanced to support long-term system evolution.

Table 15: Maintainability Requirements

ID	Requirement Description
MAI-01	Code must follow a modular structure (separate services/controllers) and include unit tests.
MAI-02	Documentation (API docs, SRS, deployment instructions) must be kept up to date in the repo.
MAI-03	The application must support smooth upgrades with migration scripts for DB changes.

Chapter 4 Design and Architecture

4 Design and Architecture

The design and architecture of the FYP Portal define how different components of the system interact to support academic workflows such as group formation, supervisor assignment, document submission, and progress tracking. The architecture ensures security, scalability, and smooth communication between the frontend, backend, and database layers. This chapter explains the structural design, data representation, process flows, and the models that guide the system's functionality.

4.1 System Architecture

The architecture of the FYP Portal follows a modular client–server model that supports role-based access, secure communication, and efficient handling of academic workflows. The system consists of a web-based frontend, developed using React.js, which provides individual dashboards for Students, Supervisors, the HOD, and Admin users. This separation ensures that each user interacts only with the features relevant to their responsibilities. The client communicates with the backend through RESTful APIs, allowing smooth data exchange and scalable operations.[8]

The backend, developed using Node.js and Express, contains the core business logic of the system. It manages authentication, group creation, supervisor assignment workflows, document submissions, notification services, and administrative configurations. The backend also implements validation rules to enforce institutional policies such as group size limits, supervisor capacities, submission deadlines, and access control. Real-time functionalities, such as immediate notifications for supervisor requests, approval decisions, or new submissions, are handled through WebSockets or a lightweight push-notification module.

Overall, the architecture promotes scalability, security, and maintainability. Each layer operates independently, enabling the system to grow as user demand increases while maintaining efficient and uninterrupted academic workflows.

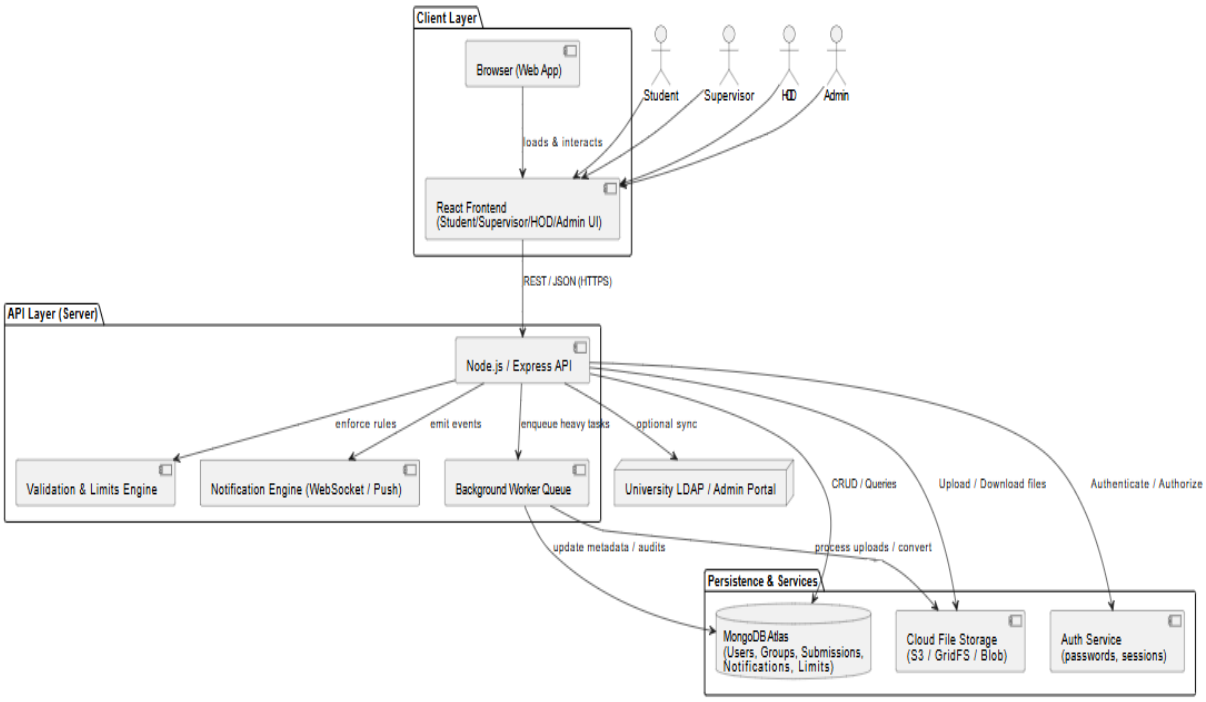

Figure 2: System Architecture

4.1.1 Layered Architecture Approach

The FYP Portal is designed using a layered architecture approach that clearly separates the system into distinct layers, each responsible for a specific set of functionalities. The presentation layer handles all user interactions through a web-based interface, allowing students, supervisors, HODs, and administrators to interact with the system according to their roles. This layer focuses solely on displaying data, collecting user input, and presenting system feedback, without embedding any business logic. By isolating the user interface from backend operations, the system ensures that interface updates or design changes do not affect core system functionality.

The application logic layer resides at the backend and serves as the central control unit of the portal. It processes user requests, applies validation rules, enforces academic policies, and coordinates data exchange between the presentation and data layers. The data layer is responsible for persistent storage, managing user records, group information, submissions, and notifications in a structured manner. This layered separation improves maintainability, simplifies debugging, and enables independent scaling of components, making the system robust and adaptable to future academic or institutional requirements.

4.1.2 Role-Based Access and Security Design

Role-based access control is a fundamental architectural principle of the FYP Portal, ensuring that each user interacts only with functionalities relevant to their assigned role. Students, supervisors, HODs, and administrators are authenticated and authorized through controlled access mechanisms that restrict actions based on role permissions. For example, students cannot access administrative controls, supervisors cannot modify system-wide rules, and only the HOD can view departmental-level summaries. This structured access control prevents unauthorized operations and protects sensitive academic data.

Security is enforced across both frontend and backend layers. At the frontend level, role-based rendering ensures that restricted options are not visible to unauthorized users. At the backend level, every request is validated to confirm the user's role and session authenticity before execution. This dual-layer enforcement protects against unauthorized access, data leakage, and misuse of academic records. By integrating access control into the architectural design, the FYP Portal maintains confidentiality, integrity, and accountability throughout the Final Year Project lifecycle.

4.1.3 Scalability and Maintainability Considerations

The architecture of the FYP Portal is intentionally designed to support scalability as the number of users, projects, and academic activities grows. Modular backend services allow individual components, such as user management, submission handling, or notifications, to be scaled independently based on system load. This ensures that increased activity during peak academic periods does not degrade system performance or reliability. The architecture also supports future integration of additional modules without restructuring the core system.

Maintainability is achieved through clear separation of concerns and standardized coding practices across the system. Each module performs a specific responsibility, making updates, bug fixes, and enhancements easier to implement and test. Documentation, consistent data models, and reusable services further reduce maintenance overhead. This architectural focus ensures that the portal remains sustainable over multiple academic cycles while accommodating evolving departmental needs.

4.2 Data Representation

The data model of the FYP Portal is designed to accurately reflect the academic processes involved in Final Year Project management. Users are stored along with their roles, credentials, and profile metadata. Students may belong to groups, which function as collaborative units for project development. Groups have associated members and are linked to supervisors through supervisor-assignment records that store request status, timestamps, and approval responses.

Submissions represent uploaded documents, containing essential metadata such as file name, file type, description, uploader identity, associated group, and review status. Notifications are generated automatically based on user actions—for example, when a supervisor receives a request, when a submission is reviewed, or when the admin updates system policies. Administrative settings, such as maximum group size or supervisor capacity, are stored in a centralized configuration table to ensure that system-wide constraints remain consistent.

The Entity–Relationship model illustrates these interactions by defining clear connections between Users, Groups, Submissions, Supervisor Assignments, Notifications, and Administrative Policies. This structured representation ensures data integrity and provides a reliable backbone for dashboard rendering, progress evaluation, and academic recordkeeping within the institution.

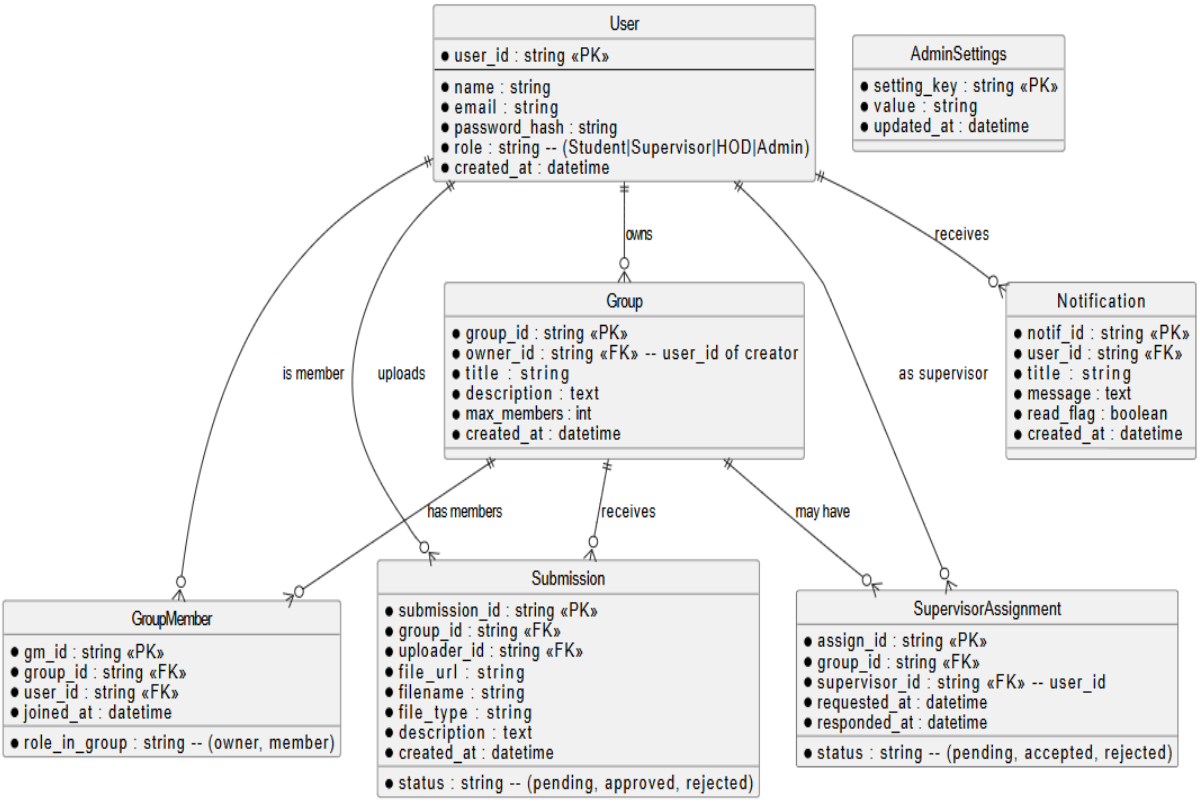

Figure 3: ERD Diagram

4.2.1 Entity Design and Data Normalization

The data entities in the FYP Portal are carefully designed to reflect real-world academic structures while avoiding redundancy and inconsistency. Core entities such as User, Group, Submission, and Notification are defined with clear attributes that store only essential information. Data normalization techniques are applied to eliminate duplicate records and

ensure that each piece of information is stored in a single, authoritative location. This design reduces data anomalies and improves overall data reliability.

Normalized entity design also simplifies updates and ensures accuracy across the system. For example, when a supervisor's details are updated, the change is reflected automatically across all related groups and submissions without manual intervention. This structured approach supports accurate dashboards, reliable progress tracking, and consistent academic records, which are essential for supervision, evaluation, and departmental reporting.

4.2.2 Relationship Mapping Between Academic Entities

Relationships between entities are central to the functioning of the FYP Portal. Students are linked to groups, groups are associated with supervisors, and submissions are tied to both groups and reviewers. These relationships enable the system to present accurate and contextual information across dashboards for all roles. For example, a supervisor can view all groups assigned to them, while the HOD can see all groups under each supervisor.

Clear relationship mapping ensures seamless navigation between academic data points and supports system automation. Notifications, approvals, and progress updates rely heavily on these relationships to reach the correct recipients. By explicitly defining these connections within the data model, the portal ensures transparency, traceability, and effective coordination across all stages of the Final Year Project process.

4.2.3 Data Integrity and Consistency Enforcement

Data integrity is enforced in the FYP Portal through validation rules and controlled database operations. Every action, such as group creation, supervisor assignment, or document submission, is verified before being committed to storage. This prevents invalid states such as duplicate group memberships, exceeded supervisor capacity, or unauthorized submissions. Consistency checks ensure that academic rules are applied uniformly across the system.

Consistency enforcement also supports long-term academic recordkeeping. Submission histories, approval logs, and notifications remain synchronized and traceable, ensuring that no data conflicts arise over time. This structured enforcement protects the credibility of academic records and supports audits, evaluations, and institutional reporting requirements.

4.3 Process Flow / Representation

The process flow of the FYP Portal demonstrates how academic interactions move through the system in a structured and automated manner. A typical workflow begins when a student registers or logs into the portal and creates or joins a project group. Once the group is formed, students send a supervisor request, which triggers automated notifications and places the request into a pending state. The supervisor reviews the group's details, accepts or rejects the request, and the system updates the assignment accordingly while notifying all group members.

Document submission forms another core workflow. A student selects a file, provides a short description, and uploads the document to the portal. The backend securely stores the file and registers a submission record in the database. Depending on institutional rules, the submission may undergo supervisor review, administrative checks, or automatic validation. At each stage, notifications ensure transparency and timely communication, keeping students informed about their project's progress.

The process flow emphasizes automation, visibility, and structured decision-making. By digitizing manual tasks such as approvals, communication, and file tracking, the system reduces administrative workload while providing students and supervisors with a clear, reliable workflow throughout the project lifecycle.

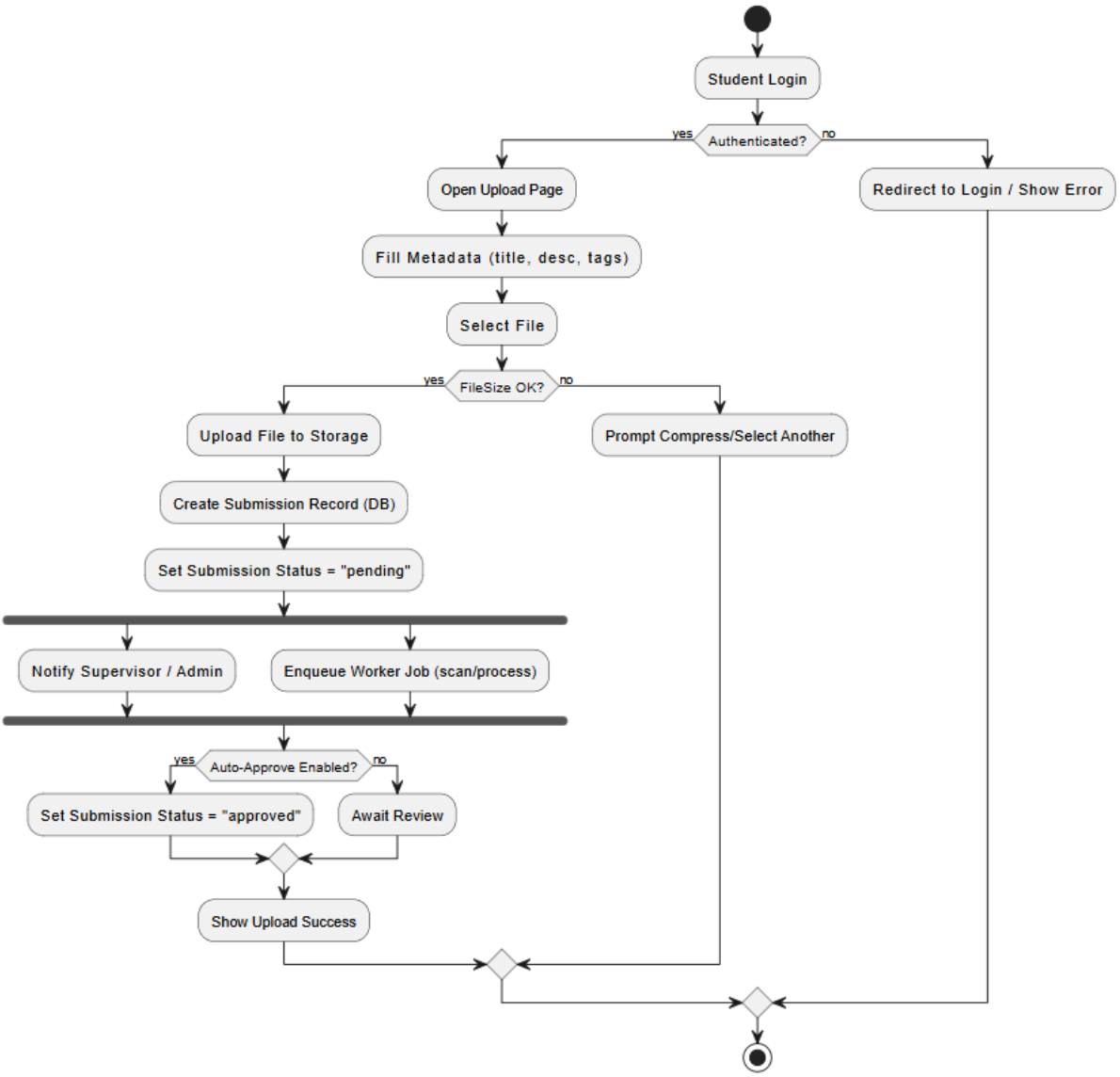

Figure 4: Process Flow Diagram

4.3.1 User Interaction Flow

The user interaction flow of the FYP Portal is designed to guide users logically through their academic tasks while minimizing confusion and errors. Each role experiences a tailored workflow that aligns with their responsibilities. Students are guided from login to group formation, supervisor requests, submissions, and progress tracking. Supervisors and administrators follow structured paths for approvals, reviews, and oversight activities.

This controlled interaction flow ensures that users perform actions in the correct sequence, preventing invalid operations. For example, students cannot submit documents before forming a group or securing a supervisor. By embedding workflow logic into the system, the portal promotes clarity, efficiency, and a smooth academic experience for all stakeholders.

4.3.2 Approval and Validation Workflow

Approval and validation workflows form the backbone of academic governance within the FYP Portal. Requests such as supervisor selection, group joining, and document submission pass through defined approval stages that ensure fairness and compliance with departmental rules. Automated validation checks eliminate the need for manual enforcement while maintaining academic discipline.

Supervisors and administrators receive requests through structured dashboards, allowing them to make informed decisions based on complete project information. Once approved or rejected, the system updates record instantly and notifies all relevant users. This workflow ensures transparency, accountability, and efficient decision-making throughout the Final Year Project lifecycle.

4.3.3 Notification and Feedback Flow

The notification and feedback mechanism ensures continuous communication among all stakeholders. Every significant action, such as request submission, approval, rejection, or document upload, triggers system-generated notifications. These alerts keep users informed in real time and reduce dependency on external communication tools.

Feedback flows are designed to close the communication loop by informing users of outcomes and next steps. Students receive supervisor responses promptly, supervisors are notified of new submissions, and the HOD is kept informed of departmental activities. This structured notification flow enhances coordination, reduces delays, and supports an organized academic environment.

4.4 Design Models

Design models describe the internal and external interactions of the system through diagrams such as sequence and class models. These models visually explain how components collaborate to complete tasks like uploading documents or assigning supervisors. They help ensure that system behaviour is logically structured and maintainable.

4.4.1 Sequence Diagram Explanation

The sequence model outlines the interaction between different system components during key actions such as document upload, supervisor assignment, or group creation. For example, in the document-upload sequence, the user initiates the upload from the web interface, which validates the session and sends both the metadata and file to the server. The backend stores the file in the designated storage module, creates a submission record in the database, and triggers notifications to supervisors or administrators. Asynchronous processes, such as file validation or background checks, are handled separately to keep the user experience responsive. This step-by-step representation demonstrates how components collaborate to complete an operation reliably and efficiently. The sequence model illustrates how different components of the FYP Portal interact with one another during key academic operations such as group creation, supervisor assignment, and document submission. It provides a time-ordered view of interactions between the user interface, backend validation logic, database, and notification services. This model helps in understanding how user requests are processed step by step and how system responses are generated in a controlled and structured manner.

For instance, during the document submission process, a student initiates the upload through the web interface, after which the system verifies the user’s session and role. The backend then

validates the submission rules, stores the document in the designated storage module, and records the submission details in the database. Once completed, relevant notifications are triggered for the supervisor or HOD. Any background tasks, such as file integrity checks or version tracking, are handled asynchronously to maintain system responsiveness. This sequence-based interaction ensures reliability, transparency, and smooth coordination across all system components.

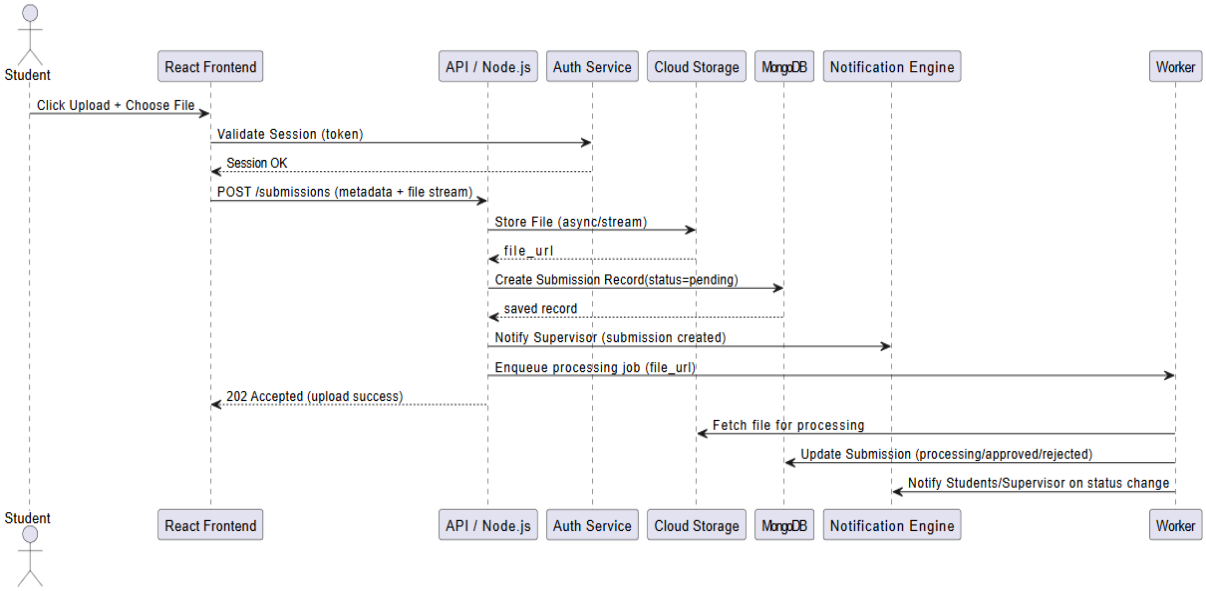

Figure 5: Sequence Diagram

4.4.2 Class Diagram Explanation

The class diagram defines the structural blueprint of the FYP Portal by presenting the system's main classes, their attributes, and their responsibilities. Core entities such as User, Group, Submission, and Notification encapsulate both data and behaviour. Service classes, like AuthService, StorageService, and ValidationEngine, handle authentication, file management, and enforcement of institutional policies. The relationships between classes show ownership, membership, aggregation, and dependency links, helping illustrate how different modules communicate within the system. This model ensures maintainability by clearly separating concerns and enabling future extension without disrupting the existing architecture.

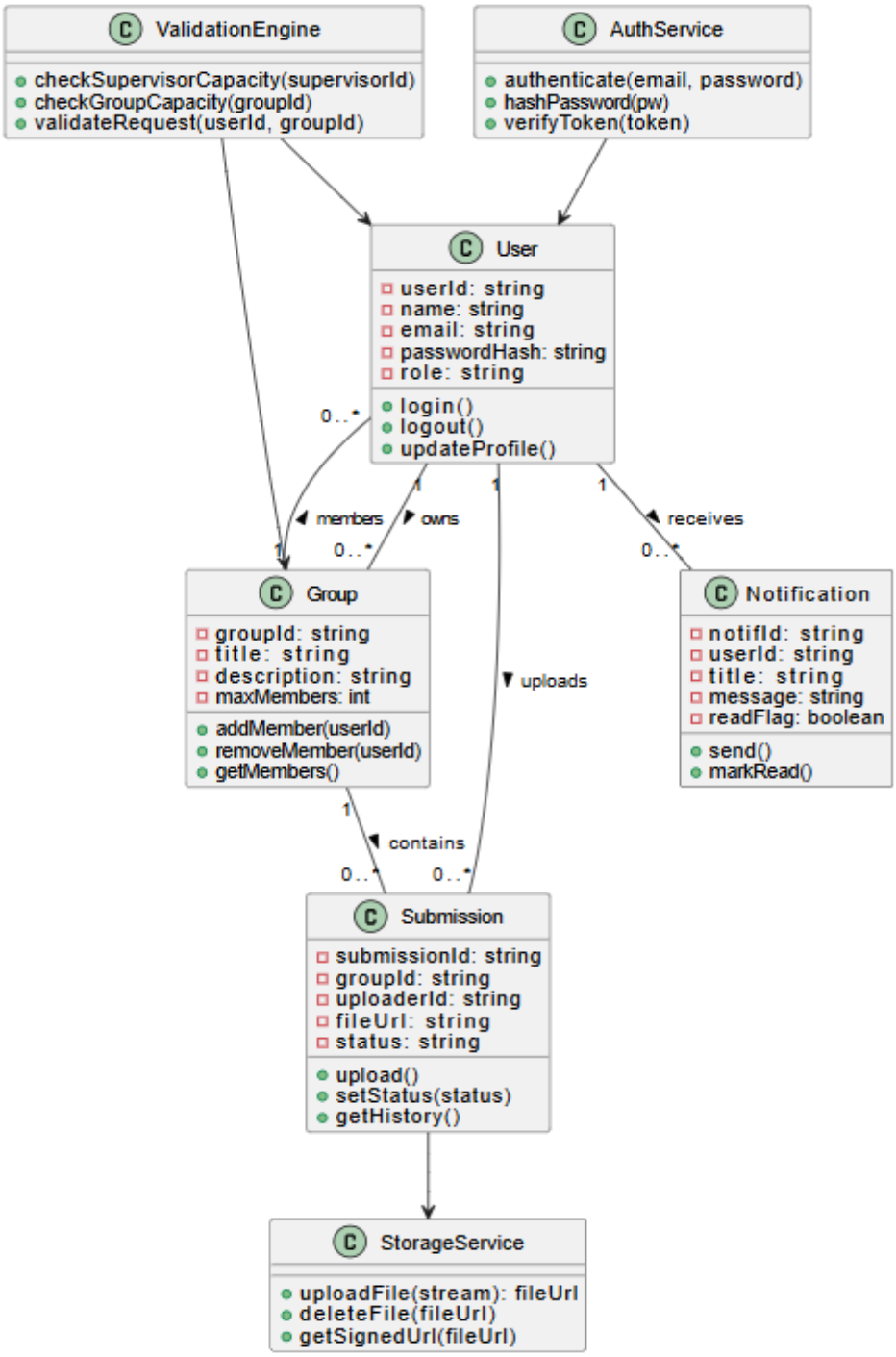

Figure 6: Class Diagram

4.4.3 Design Consistency and Extensibility

Design consistency ensures that all components of the FYP Portal follow uniform architectural and interaction principles. Consistent data handling, validation logic, and interface behaviour reduce user learning time and system complexity. This uniformity also simplifies development and maintenance by enforcing predictable system behaviour.

Extensibility is achieved by designing components that can be enhanced without disrupting existing functionality. New features such as automated evaluations, analytics dashboards, or plagiarism detection modules can be integrated into the system using existing services and data

models. This future-ready design ensures that the FYP Portal remains relevant and adaptable as academic and technological requirements evolve.

4.4.4 System Modularity and Reusability

The FYP Portal is designed with a modular structure that allows individual components to be developed, tested, and maintained independently. Each module handles a specific responsibility such as authentication, group management, submissions, or notifications, reducing interdependencies across the system. This modularity improves code reusability and simplifies future enhancements or replacements of system components. As a result, the overall architecture remains flexible, maintainable, and adaptable to evolving academic requirements.

Chapter 5 Implementation

5 Implementation

The implementation of the FYP Portal involves the coordinated development of both backend and frontend modules to deliver a unified academic workflow solution. This includes managing user roles, group creation, supervisor assignments, document submissions, messaging, and notifications. The system was built using modern web technologies to ensure security, performance, and ease of scalability. Below is an overview of the implementation approach.

5.1 Frontend Development

The frontend of the FYP Portal is developed using React.js (or your chosen frontend framework), enabling a responsive and intuitive interface for students, supervisors, HODs, and admins. Components such as forms, dashboards, submission cards, messaging panels, and notification badges are built using reusable UI modules. Role-based rendering ensures that each user sees only the functions relevant to their role. The frontend communicates with backend APIs through secure HTTP requests and displays real-time updates such as submission status, supervisor responses, and system announcements.

5.1.1 Role-Based User Interface Design

The FYP Portal frontend is designed around a role-based user interface that dynamically adapts according to the logged-in user's role. Students, supervisors, HODs, and administrators are presented with different dashboards, menus, and actions based on their assigned permissions. This ensures clarity, reduces confusion, and prevents unauthorized access to restricted features.

The frontend checks the authenticated user's role after login and conditionally renders components such as group management panels, approval screens, or administrative controls. This approach improves usability while enforcing access control directly at the presentation layer, complementing backend security mechanisms.

5.1.2 Dashboard and Navigation Structure

Each role in the FYP Portal is provided with a dedicated dashboard that serves as the primary interaction space. Student dashboards display group details, project information, submission status, and supervisor feedback. Supervisor dashboards focus on group listings, pending requests, document reviews, and communication tools, while the HOD and Admin dashboards provide high-level oversight and control features.

Navigation is designed to be simple and consistent across all roles, using a centralized menu structure and clear action buttons. This structured navigation minimizes learning time for users and ensures that key academic tasks can be performed efficiently without relying on external guidance.

5.1.3 Forms, Validation, and User Feedback

The frontend implements structured forms for actions such as login, group creation, supervisor requests, and document uploads. Client-side validation ensures that required fields are completed, formats are correct, and invalid inputs are prevented before sending data to the backend. This reduces server load and improves user experience.

Immediate feedback is provided through alerts, confirmation messages, and visual indicators such as status badges. For example, users receive instant confirmation when a request is submitted or rejected, ensuring transparency and reducing uncertainty during critical academic workflows.

5.2 Backend Development

The backend is built using Node.js & Express.js (or your backend choice), responsible for handling authentication, user roles, group workflows, supervisor approval logic, and document management. A MongoDB database stores structured data such as users, projects, submissions, messages, and notifications. Authentication is implemented using JWT tokens, ensuring secure login and session handling. Admin-specific APIs allow creation of users, enforcement of group limits, and monitoring of system usage.[9] The backend ensures data consistency by validating all operations, such as preventing duplicate groups or conflicting supervisor assignments.

5.2.1 Authentication and Authorization Management

The backend manages authentication using secure mechanisms that verify user credentials and generate session tokens upon successful login. Each request sent from the frontend includes authentication information, allowing the backend to identify the user and enforce role-based access control. This ensures that users can only perform actions permitted for their role.

Authorization logic is implemented at the API level, preventing unauthorized access to sensitive operations such as modifying group limits or viewing departmental data. This layered security approach ensures data protection and compliance with academic integrity requirements.

5.2.2 Data Modelling and Database Management

The backend uses a structured database schema to manage users, groups, supervisors, submissions, messages, and notifications. Relationships between entities are clearly defined, allowing efficient retrieval of data such as group members, supervisor assignments, and submission histories. This structured approach supports consistency and scalability.

All academic records are stored with timestamps and references, ensuring traceability throughout the project lifecycle. The database design supports future extensions such as analytics, reporting, and automated evaluations without requiring major architectural changes.

5.2.3 Business Logic and Rule Enforcement

Core academic rules are enforced through backend business logic rather than relying solely on frontend checks. This includes validating group size limits, supervisor capacity, duplicate request prevention, and submission eligibility. Every operation is verified before being committed to the database.

By centralizing rule enforcement in the backend, the system ensures consistent behaviour across all users and interfaces. This prevents manipulation, reduces errors, and guarantees fairness in supervisor allocation and group formation.

5.3 Algorithms

The FYP Portal incorporates several backend algorithms designed to ensure efficient workflow automation, accurate validation, and seamless interaction between students, supervisors, and administrators.

5.3.1 Supervisor Assignment Validation Algorithm

- Triggered when a student submits a supervisor request.
- Checks if the supervisor has available capacity according to department rules.
- Verifies group eligibility: complete profile, valid group size, correct project type.
- Supervisor receives notification and approves/rejects request.

- System updates group status and sends confirmation to all group members.

5.3.2 Document Versioning & Submission Tracking Algorithm

- Each submission (SRS, SDS, Proposal, Chapters, Final Report) receives a unique version ID.
- When a new file is uploaded, older versions are archived instead of deleted.
- Metadata stores timestamps, uploader, supervisor remarks, and evaluation results.
- Ensures supervisors always view the latest version without losing submission history.
- Enables automated reminders if deadlines are approaching or missed.

5.3.3 Group Formation and Join Validation Algorithm

This algorithm is triggered whenever a student attempts to create or join a project group. It verifies conditions such as whether the student is already part of another group, whether the group exists, and whether the maximum group size has been reached. Only if all conditions are satisfied is the action approved.

The algorithm ensures that academic rules are enforced automatically, eliminating manual intervention. By validating group membership in real time, the system prevents conflicts and ensures accurate project records across the department.

5.3.4 Supervisor Request Handling Algorithm

When a student group submits a supervisor request, this algorithm checks whether the supervisor has remaining supervision capacity and whether the group meets eligibility criteria. If valid, the request is forwarded to the supervisor's dashboard for approval or rejection.

Once the supervisor responds, the algorithm updates the group's status and notifies all relevant users. This structured process ensures transparency, prevents overloading supervisors, and maintains balanced supervision across the department.

5.3.5 Notification and Event Trigger Algorithm

The notification algorithm operates in response to key system events such as group creation, supervisor responses, document submissions, and administrative announcements. Each event triggers the generation of role-specific notifications that are delivered to relevant dashboards in real time.

This algorithm ensures timely communication between stakeholders and reduces reliance on informal communication channels. By automating alerts, the system keeps all users informed and engaged throughout the FYP lifecycle.

5.4 External APIs

Below is the External APIs Table for the FYP Portal system.

Table 16: External APIs

Name of API	Description of API	Purpose of Usage	Function/Class Used In
JWT Authentication API	Secure token-based authentication service for login and session management.	User login, logout, role-based access, session validation.	AuthService (login(), verifyToken(), logout ())

MongoDB Atlas API	Cloud-hosted NoSQL database service.	Stores users, groups, supervisors, submissions, messages, and notifications.	DatabaseService (createGroup(), saveSubmission(), getUserData())
Cloud File Storage API	Secure file storage with versioning and access control.	Upload and retrieve project files such as SRS, SDS, proposals, and reports.	FileUploadManager (uploadDocument(), getFileURL())
Email Notification API	Automated email delivery system.	Sends email alerts for supervisor requests, submission approvals, deadlines.	NotificationService (sendEmail(), scheduleReminder())
Push Notification API	Browser and device-level notification delivery.	Notify users of supervisor responses, group updates, and new announcements.	NotificationService (subscribeUser(), sendPush())

5.5 User Interface

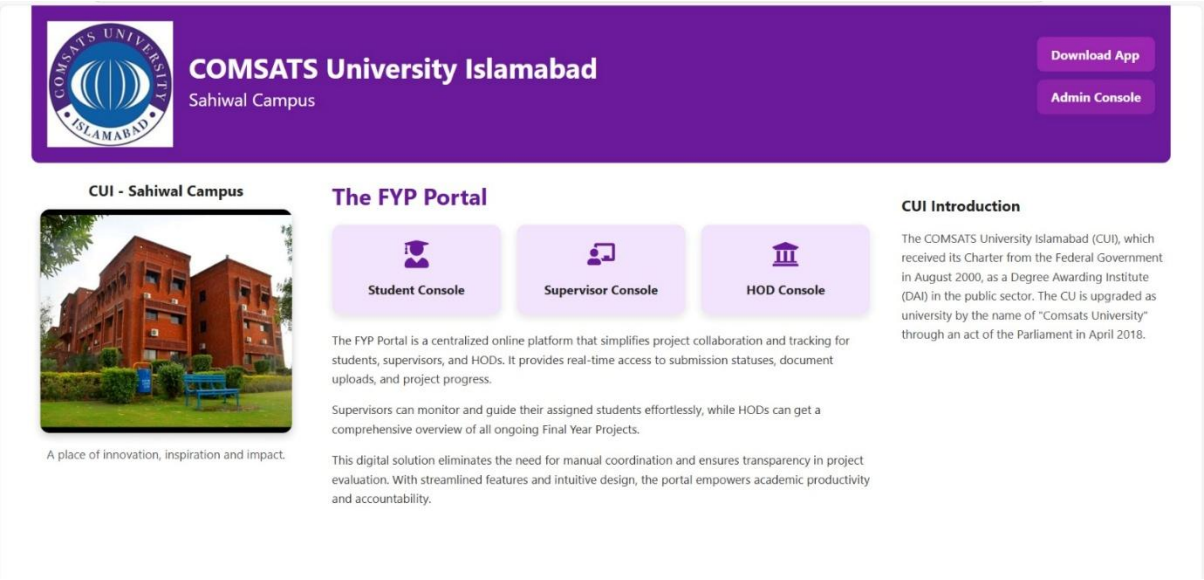

Figure 7: Home Page

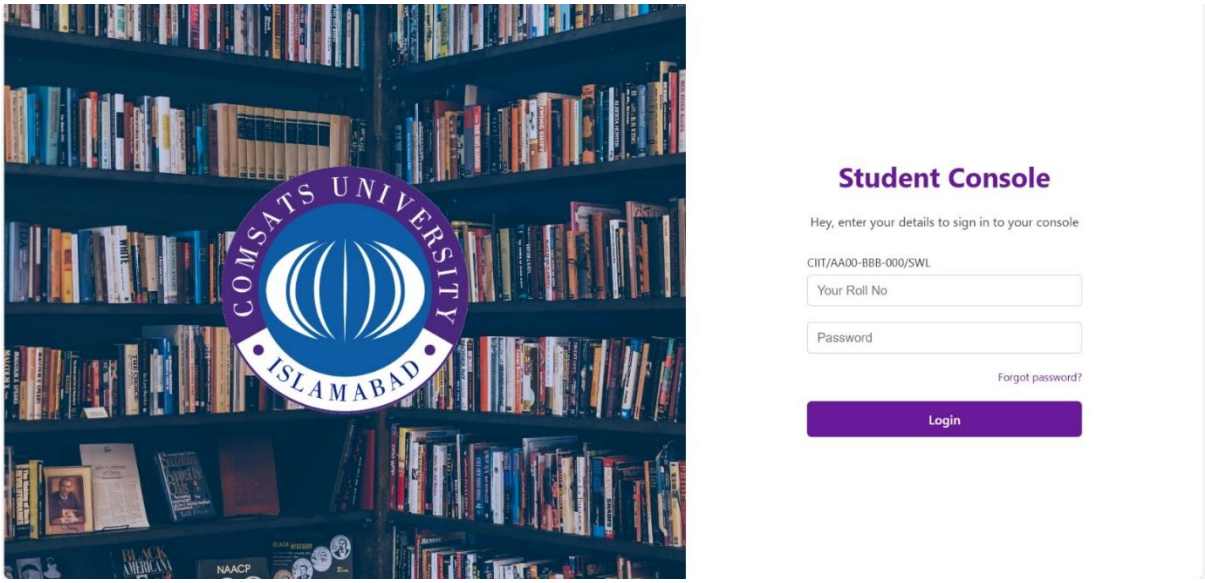

Figure 8: Student Login

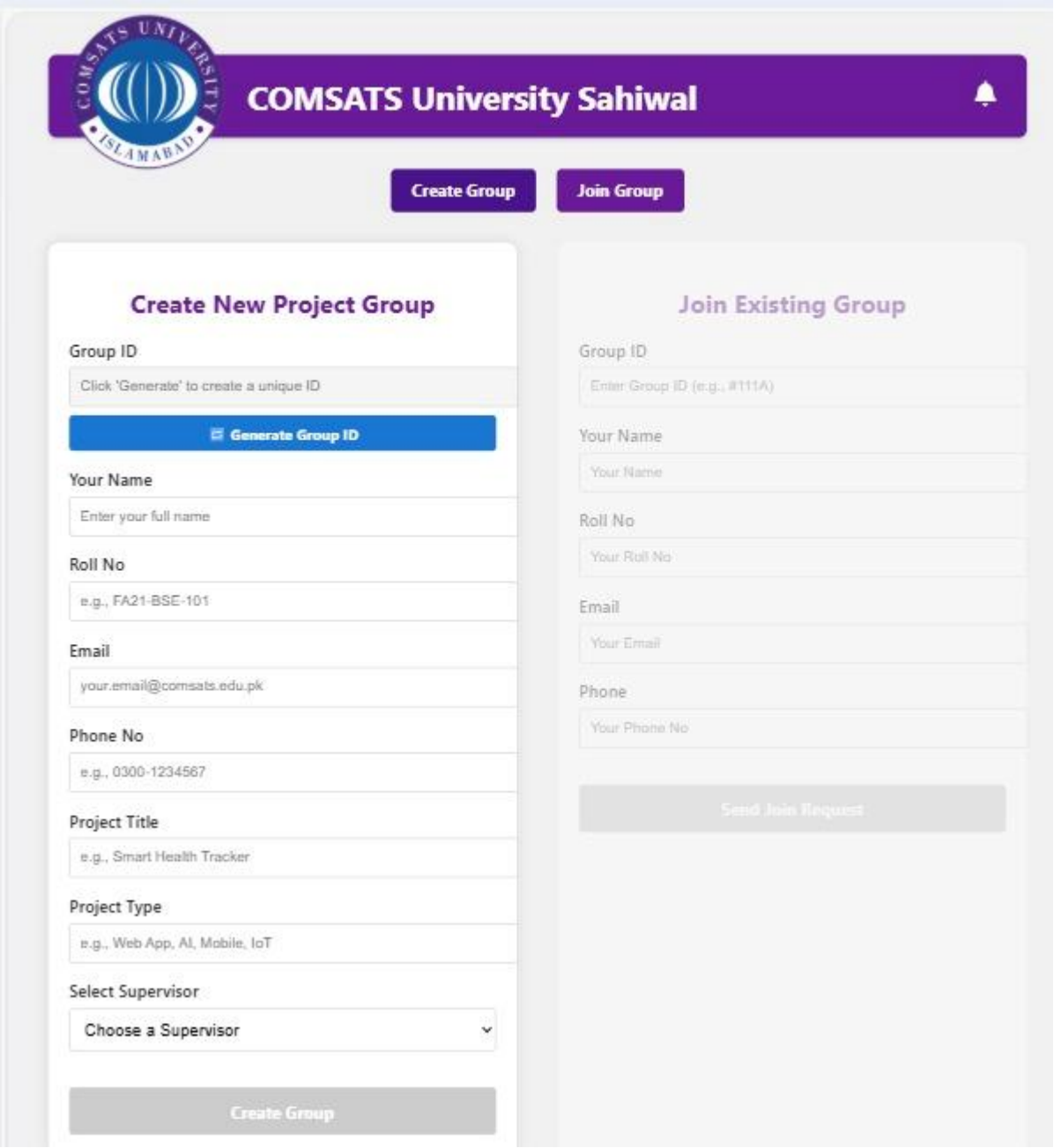

Figure 9: Create Group Page

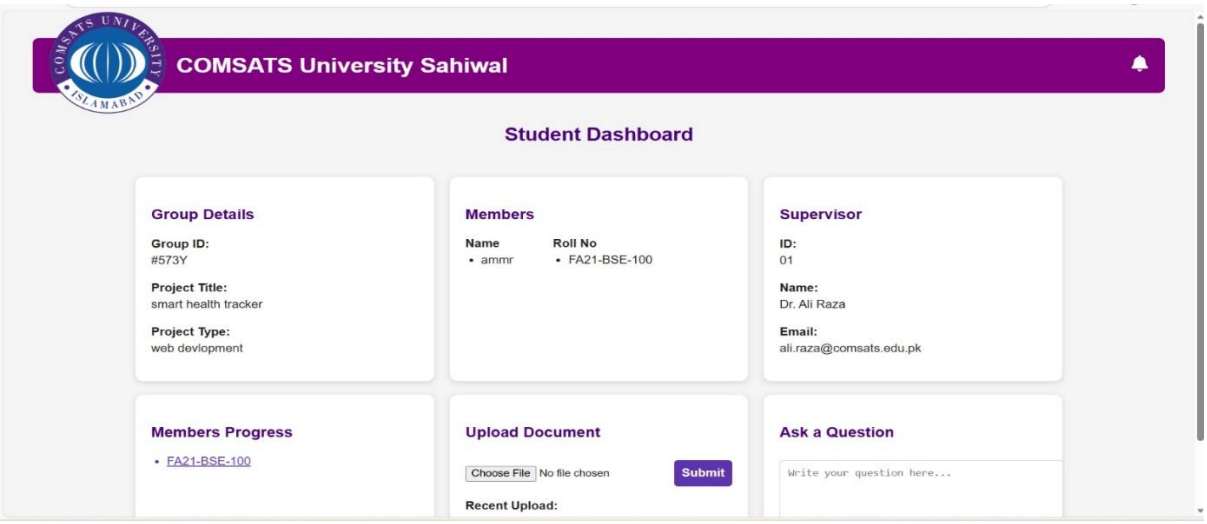

Figure 10: Student Dashboard

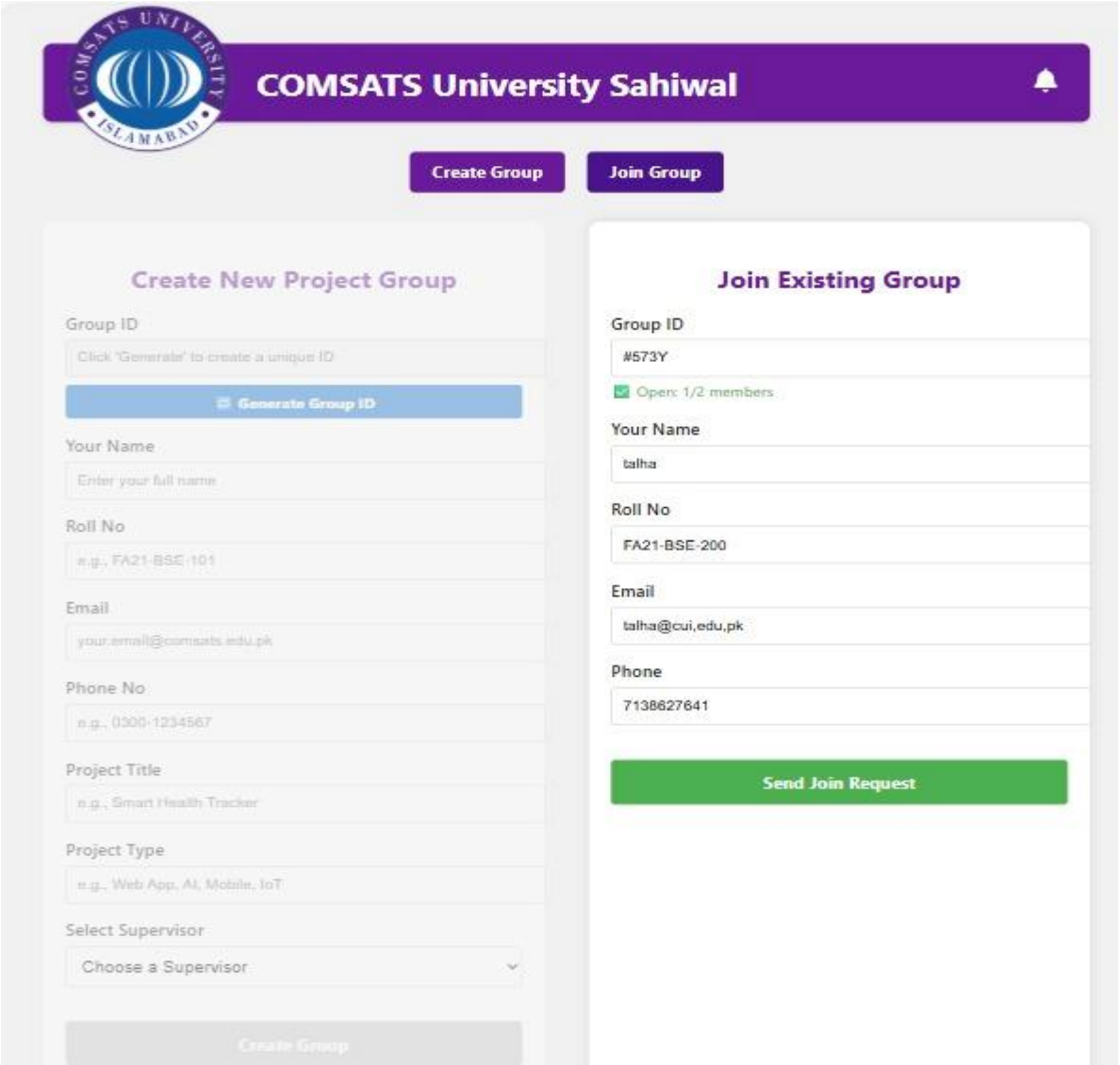

Figure 11: Join Group Page

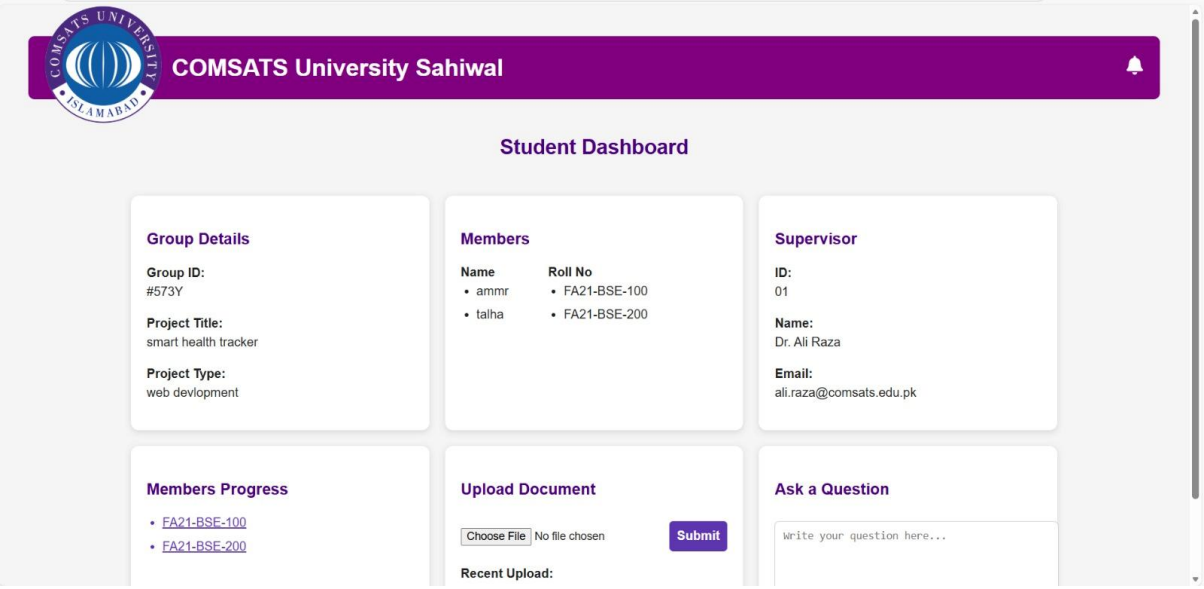

Figure 12: Updated Dashboard after joining of new student

The FYP Portal begins with a centralized home (landing) page that serves as the entry point for all users. From this page, users can choose their respective role, Student, Supervisor, HOD, or Admin. This role-based entry ensures that each user is redirected to the correct authentication and functional environment according to their responsibilities within the Final Year Project process.

When a student selects the Student Console, they are directed to the student login page, where authentication is performed using their university roll number and password. This secure login mechanism ensures that only registered students can access project-related features and data.

After successful login, if the student has not yet joined or created a project group, the system redirects them to the Group Management screen. At this stage, the student can either create a new project group or join an existing group. While creating a group, the student generates a unique Group ID, enters project details such as project title, project type, and selects a supervisor. This information is stored centrally and becomes the base record for the project.

Once the group is created, the Student Dashboard is automatically displayed. This dashboard acts as the main working area for students. It shows essential information including Group Details, Project Title, Project Type, assigned Supervisor information, and current group members.[10] At this point, the dashboard reflects a single-member group.

When another student logs into the system and chooses the Join Group option, they enter the same Group ID and submit a join request. Upon successful joining, the system updates the group record in real time. The dashboard is refreshed automatically to reflect the updated group composition, now displaying two students under the Members section.

The updated Student Dashboard now serves as a collaborative workspace for both students. It displays shared project information, member progress, document upload functionality, and a communication section for asking questions or interacting with the supervisor. This real-time synchronization ensures transparency and smooth coordination among group members.

Overall, this flow demonstrates how the FYP Portal automates the complete student-side project lifecycle, from login and group formation to collaboration and progress tracking, eliminating manual coordination and ensuring an organized, transparent Final Year Project management process.

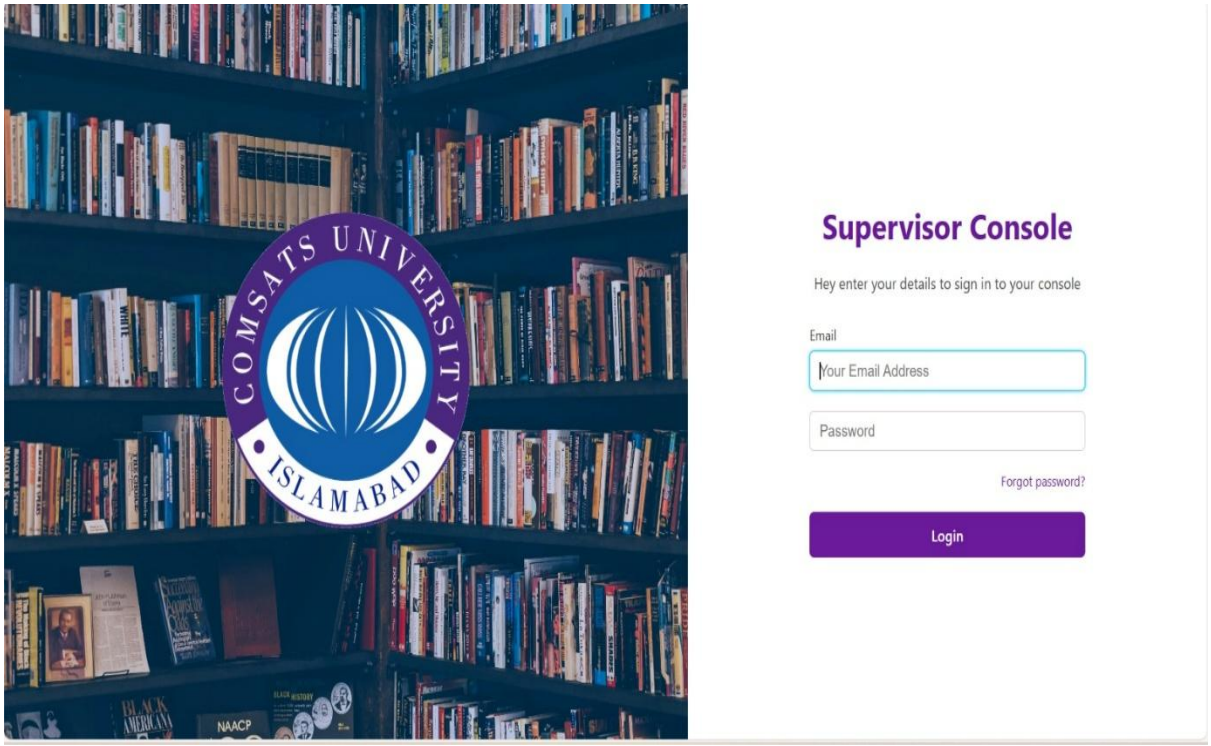

Figure 13: Supervisor Login

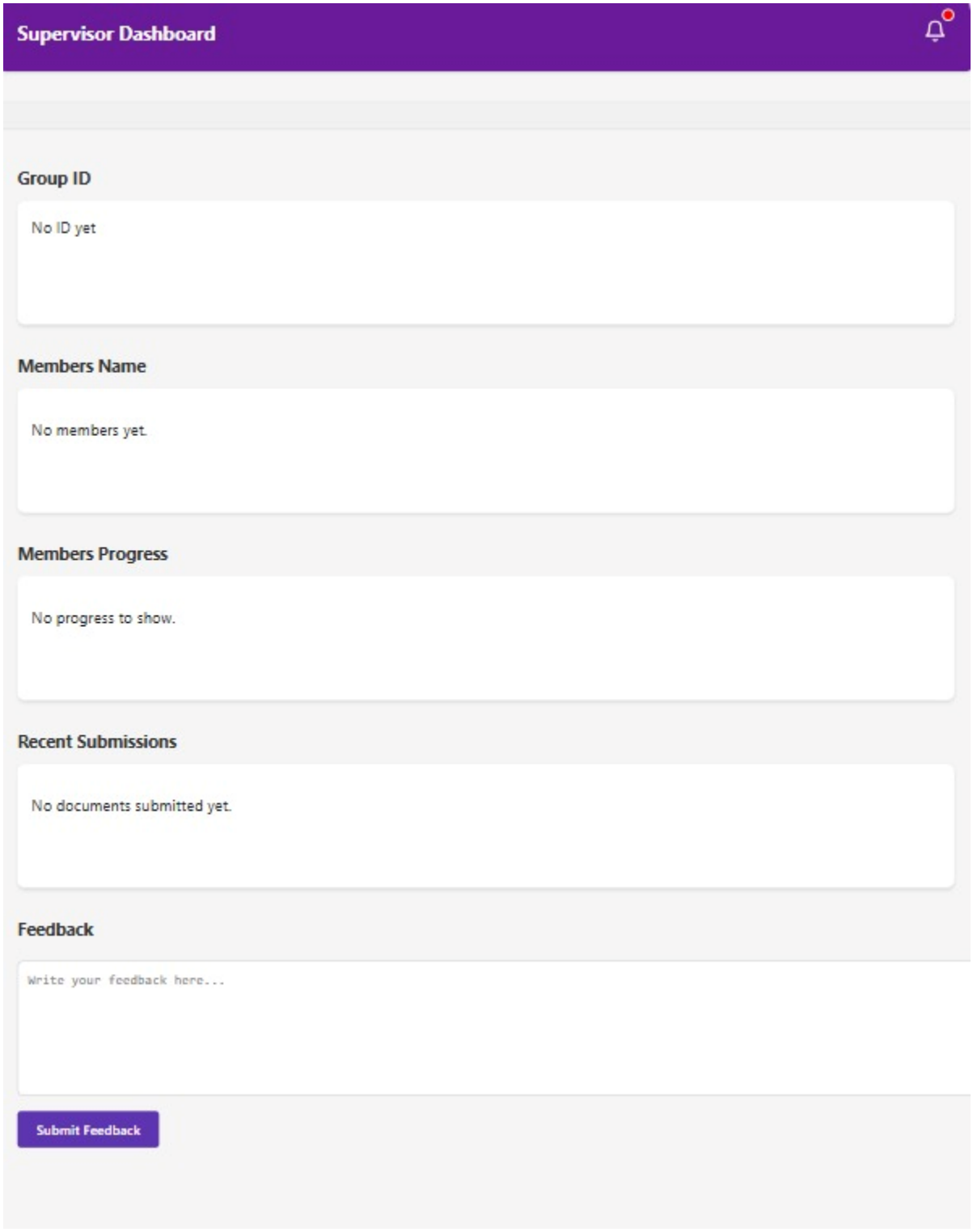

Figure 14: Supervisor Dashboard (No one is registered)

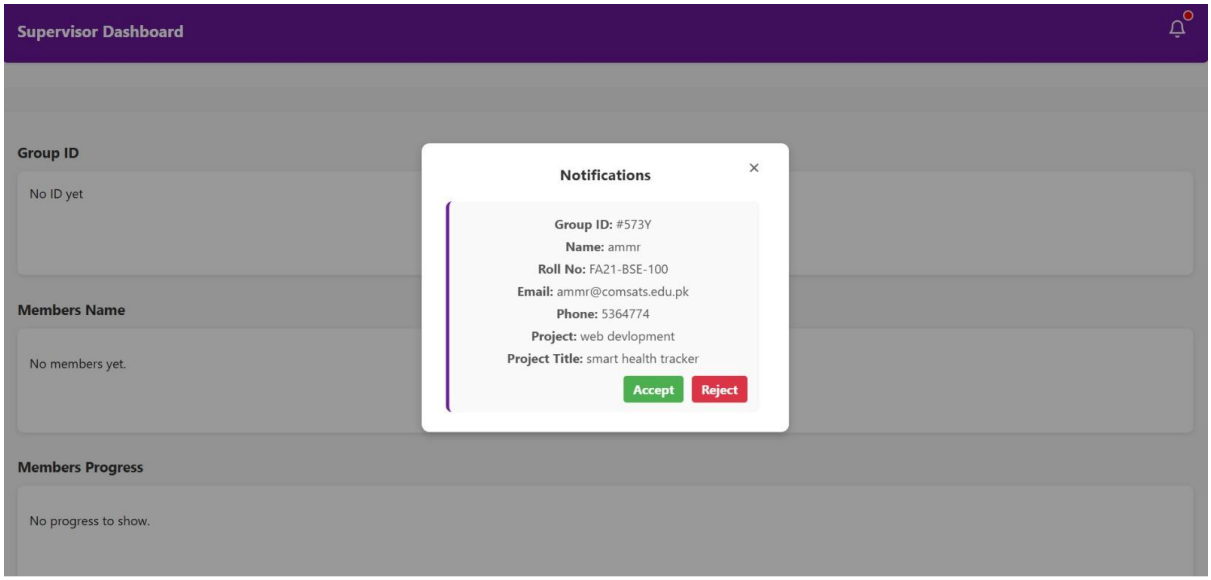

Figure 15: Registration notification to supervisor

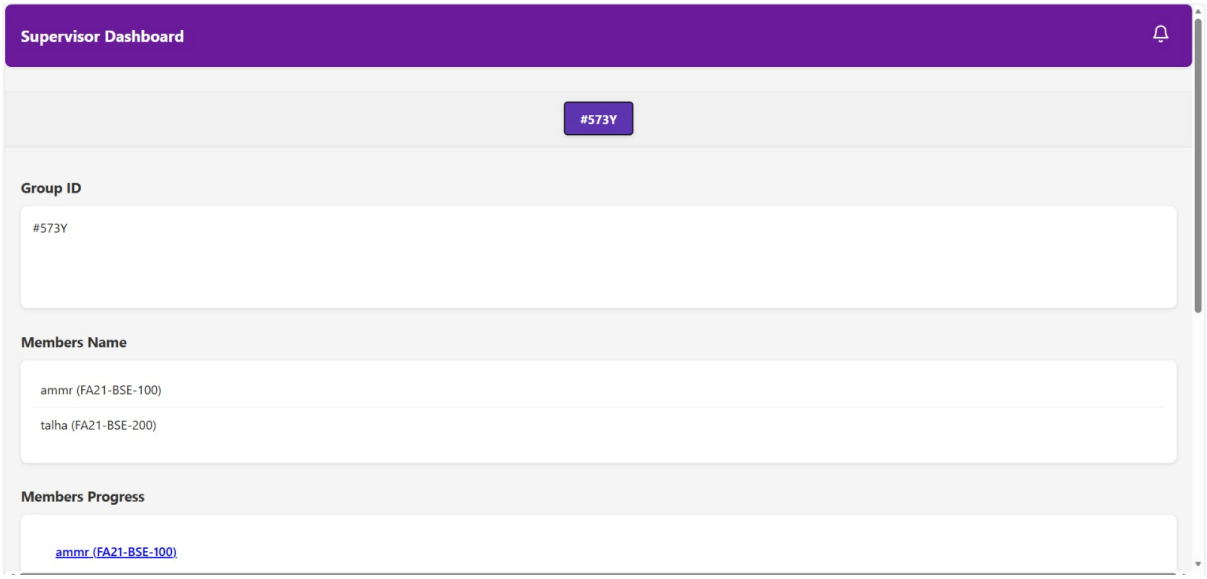

Figure 16: Updated dashboard after registration

The supervisor module of the FYP Portal is designed to facilitate effective monitoring and management of Final Year Projects while ensuring smooth coordination between supervisors and students. The process begins when a supervisor logs into the system using official credentials through the supervisor console. After successful authentication, the supervisor is redirected to the dashboard, which serves as a centralized interface for viewing project groups, student information, progress updates, and submissions.

When no student group is registered under a supervisor, the dashboard initially displays an empty state. In this condition, the group ID, members list, progress section, and recent submissions remain unavailable, clearly indicating that no active project groups exist. This helps supervisors immediately understand the current status without confusion and ensures transparency from the start of the supervision process.

As soon as a student creates a project group and selects a supervisor, the system sends a join request to the respective supervisor. The supervisor receives a real-time notification on the dashboard informing them that a student has requested to join a specific group. This notification mechanism eliminates the need for manual communication and ensures timely awareness of new group requests.

Upon receiving a request, the supervisor can review the student's details and either accept or reject the request. This approval process allows supervisors to maintain control over the composition of their project groups and ensures that only appropriate students are registered under their supervision. Once the request is approved, the student becomes an official member of the group, and the dashboard updates automatically.

After group registration, the dashboard displays complete project information, including the group ID, project title, project type, and a list of registered members with their roll numbers. The members' progress section becomes active, allowing the supervisor to track individual contributions and overall project advancement. Any documents uploaded by students appear under recent submissions, enabling structured review and evaluation.

The system also supports continuous communication through a feedback section where supervisors can provide comments, suggestions, and guidance directly related to the project. This ensures proper documentation of feedback and promotes a structured supervision workflow. Additionally, the system enforces a limit of a maximum of three project groups per supervisor, ensuring balanced workload distribution and maintaining the quality of supervision throughout the Final Year Project lifecycle.

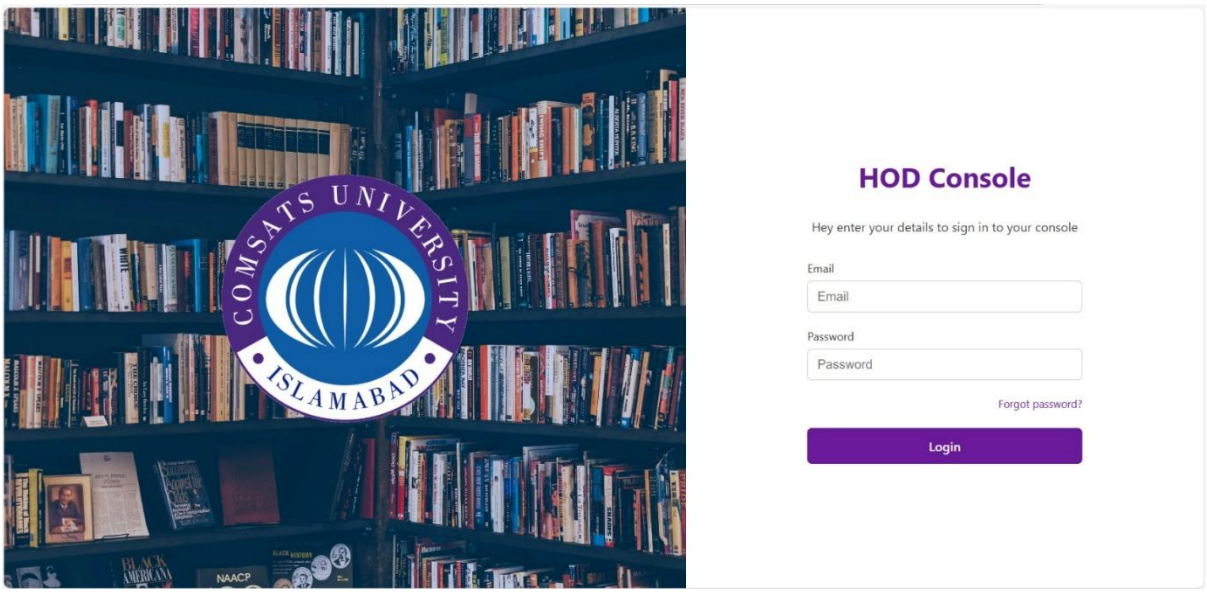

Figure 17: HoD Login page

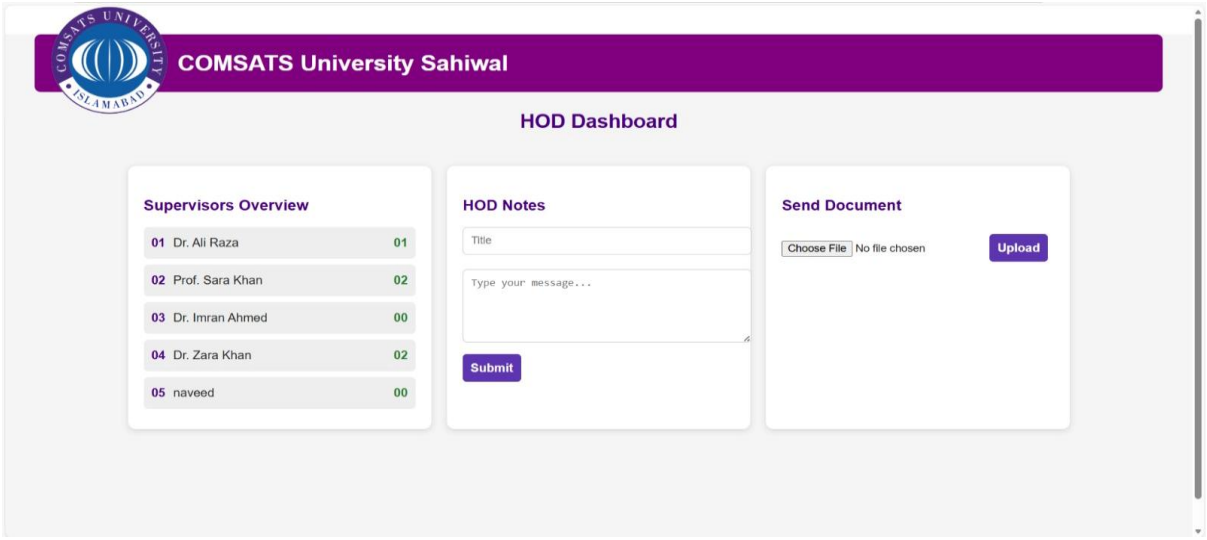

Figure 18: HoD Dashboard

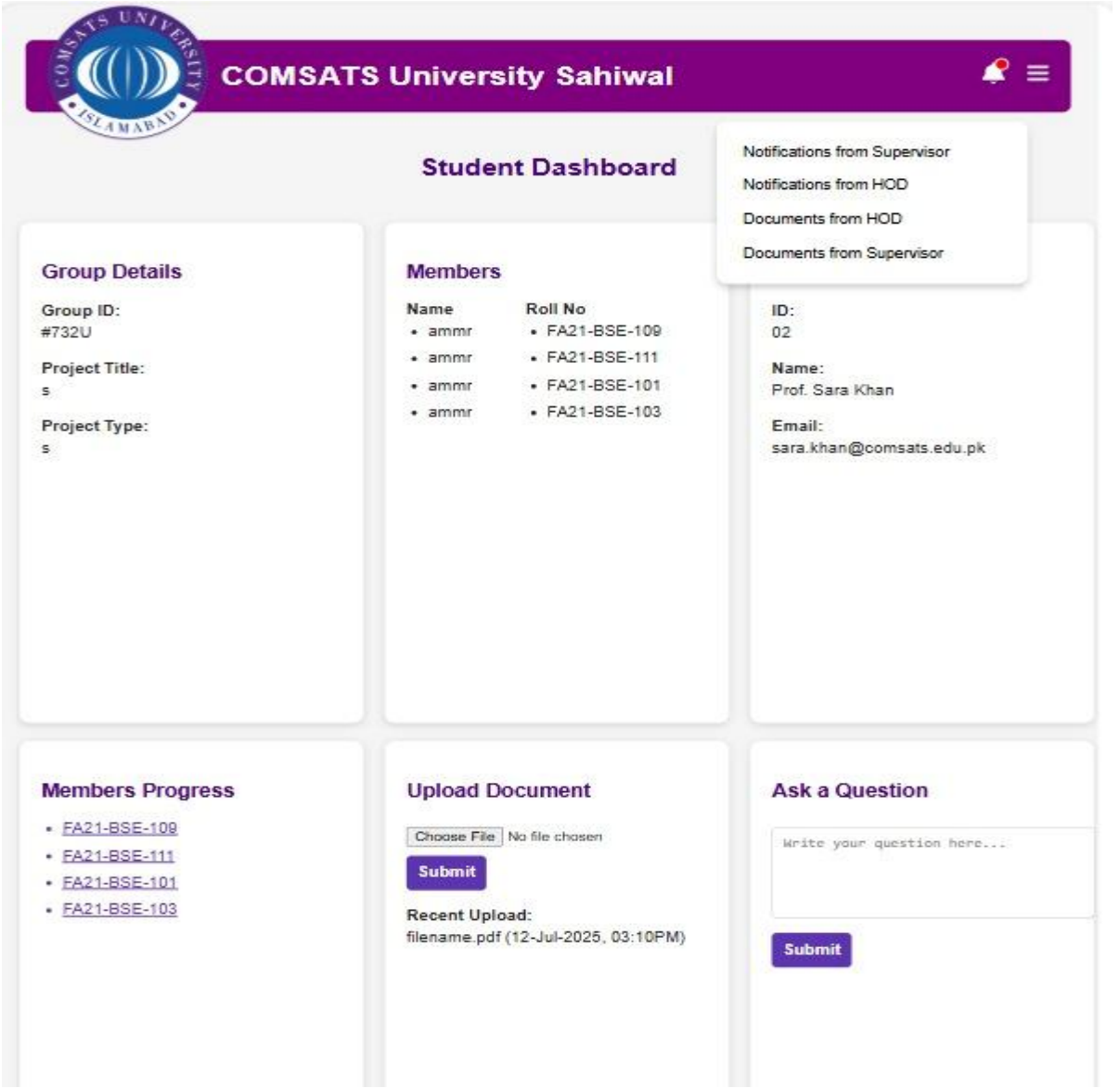

Figure 19: HoD notification showing on student dashboard

The HOD accesses the system through a secure login interface, similar to students and supervisors, ensuring role-based access control across the portal. Once logged in, the HOD is redirected to the HOD dashboard, which provides a centralized, high-level view of all academic activities related to Final Year Projects within the department. This dashboard is designed to support administrative oversight rather than day-to-day project execution, allowing the HOD to monitor progress without interfering with supervisor, student workflows.

On the HOD dashboard, a consolidated overview of all supervisors is displayed along with the number of groups currently registered under each supervisor. This enables the HOD to quickly assess workload distribution, identify supervisors who are over- or under-assigned, and ensure compliance with departmental policies such as the maximum number of groups allowed per supervisor. By clicking on a supervisor entry, the HOD can view the list of project groups associated with that supervisor, including group IDs, project titles, and current statuses.

The HOD also has the authority to communicate officially with supervisors and students through notes and document sharing. Using the HOD notes and document upload functionality, important announcements, guidelines, templates, or evaluation criteria can be shared centrally. Once uploaded, these documents are propagated to the relevant dashboards, and students receive HOD notifications directly within their student dashboard notification menu, ensuring timely and transparent communication.

From the student perspective, HOD-issued notifications and documents appear alongside supervisor communications, clearly labelled to distinguish their source. This ensures students are always aware of official instructions, deadlines, or policy updates issued at the departmental level. The integration of HOD notifications into the student dashboard removes the dependency on external communication channels and minimizes the risk of missed information.

Overall, the HOD module completes the hierarchical workflow of the FYP Portal by providing governance, visibility, and control at the departmental level. It ensures alignment between students, supervisors, and departmental leadership, promotes transparency in project allocation and monitoring, and supports efficient academic management of Final Year Projects across the university.

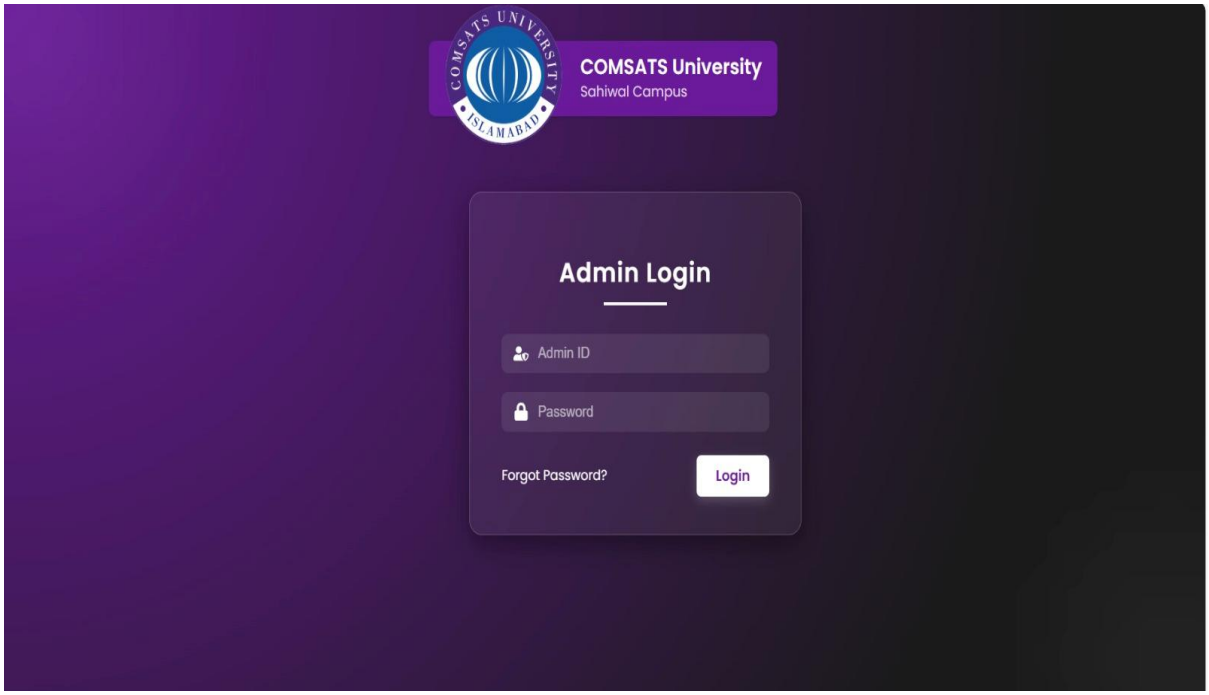

Figure 20: Admin Login

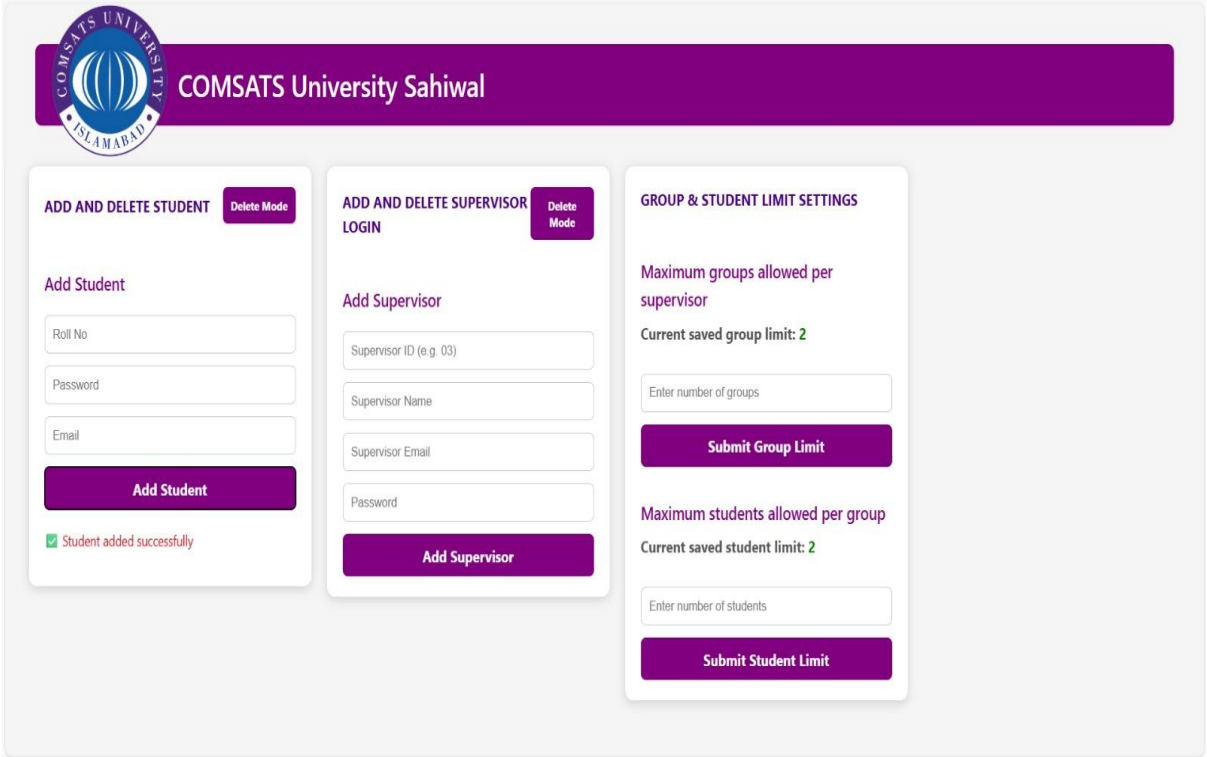

Figure 21: Adding Student or supervisor in the database

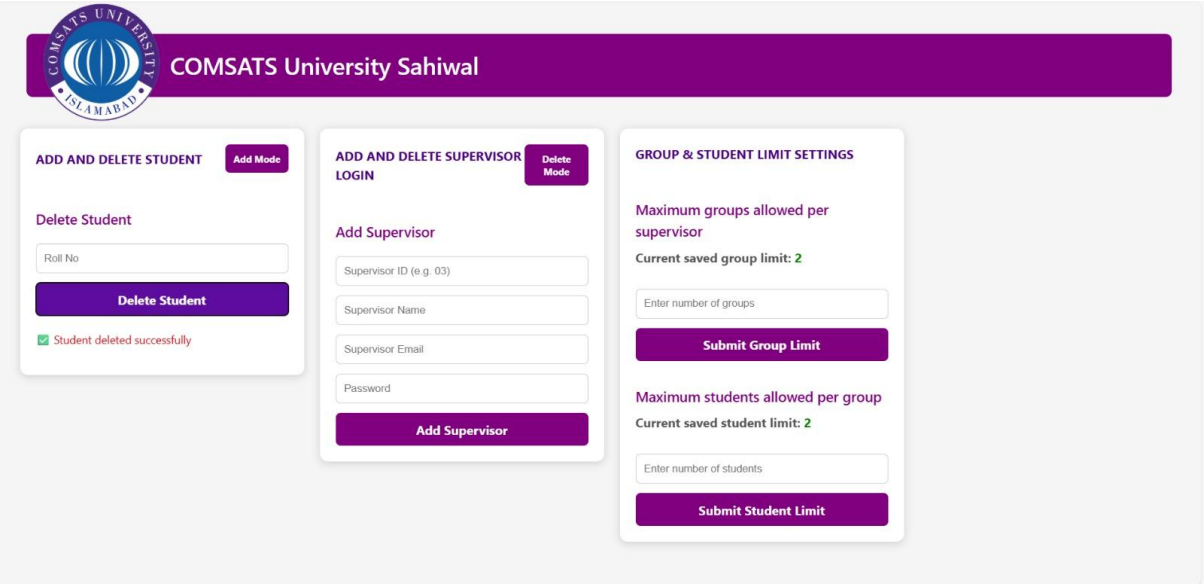

Figure 22: Delete Student and supervisor from database

The admin module of the FYP Portal serves as the central control unit responsible for managing the entire system's academic users and configuration rules. Through a secure admin login, authorized personnel can access a dedicated dashboard that allows direct interaction with the system's core database. This ensures that only verified students and supervisors are able to participate in the Final Year Project process, maintaining institutional control and data integrity from the very beginning.

Using the admin dashboard, the administrator can add new students by entering essential credentials such as roll number, email, and password, making them eligible to log in to the student console. Similarly, supervisors can be registered by providing their unique supervisor ID, name, email, and login credentials. These actions formally onboard users into the system, enabling smooth downstream processes such as group creation, supervision assignment, and progress tracking. Whenever a student or supervisor is successfully added, the system provides a confirmation message to ensure transparency and reduce administrative errors.

The admin also has the authority to delete or remove students and supervisors from the system when required, such as in cases of withdrawal, role changes, or data correction. By switching between add mode and delete mode, the administrator can efficiently manage records without affecting unrelated users. Upon deletion, the system immediately updates the database and displays a confirmation message, ensuring that obsolete or invalid accounts are removed and cannot access the portal further.

In addition to user management, the admin plays a critical role in defining academic constraints through configurable limits. The system allows the admin to set the maximum number of students allowed per project group and the maximum number of groups a supervisor can supervise simultaneously. These limits enforce university policies, prevent overloading supervisors, and ensure fair distribution of resources across all FYP groups. Once updated, these limits are applied system-wide in real time and reflected across student and supervisor workflows.

Overall, the admin functionality ensures governance, consistency, and scalability of the FYP Portal. By centralizing user registration, deletion, and policy enforcement, the system minimizes manual coordination, reduces administrative overhead, and guarantees that all subsequent activities, such as group formation, supervision, evaluation, and communication, operate within well-defined academic rules.

Chapter 6 Testing and Evaluation

6 Testing and Evaluation

Testing and evaluation were carried out to ensure that the FYP Portal operates correctly, efficiently, and securely across all major modules such as user authentication, group management, supervisor allocation, document uploads, messaging, and administrative functions. Both manual and automated testing methods were adopted to verify that the implemented system meets the functional and non-functional requirements defined in earlier chapters. This chapter outlines the testing approaches, results, and evaluation of system behaviour under different scenarios.

6.1 Manual Testing

During the manual testing phase, the system was thoroughly evaluated for functionality, usability, consistency, and performance. Each module of the FYP Portal was tested individually and then collectively to verify that the system behaves as expected under normal and abnormal usage conditions.

6.1.1 System Testing

System testing for the FYP Portal was performed to ensure that the complete web application worked correctly as a unified system. Since the platform supports multiple academic roles, Students, Supervisors, HOD, and Admin, the testing process focused on validating that all workflows functioned seamlessly across different modules such as group creation, supervisor assignment, document submission, evaluation tracking, messaging, and announcements. The goal of system testing was not only to check whether individual features worked but also to confirm that the portal behaved accurately when end-to-end academic processes were executed in real scenarios.

During testing, each functional requirement was validated against real-world actions performed by users. For example, when a student created a project group, the system was expected to update the database, reflect the group on the student dashboard, and enable the next steps such as requesting a supervisor. Similarly, when supervisors reviewed submissions or sent feedback, the system had to correctly update the submission history, notify the students, and ensure secure access to uploaded files.[11] All interactions between the front-end interface and the server-side logic were tested to ensure smooth communication and consistent data handling.

System testing also included validation of error handling, user-role restrictions, and data reliability. Incorrect inputs, unauthorized attempts (such as students trying to access admin pages), and missing data conditions were tested to ensure the portal responded with proper validations and security restrictions. Performance checks were also conducted to ensure that the portal could handle multiple simultaneous logins during peak submission periods. This thorough evaluation confirmed that the FYP Portal operates reliably, maintains data integrity, and fully supports the academic workflow from project creation to final submission.

6.1.2 Unit Testing

Unit testing validates each component of the portal individually to ensure it performs correctly before combining it with other modules. Each function, such as validating login inputs, creating groups, submitting documents, or sending messages, was tested with valid, invalid, and boundary-case data to confirm reliable behaviour.

Unit Testing 1: Login Validation

Testing Objective: Ensure that only valid user credentials allow login and that unauthorized access is rejected.

Table 17: Unit Testing 1

No.	Test Case	Attribute & Value	Expected Result	Result
1	LoginService.validate() with valid credentials	email: student@uni.edu password: Test#123	Redirects to dashboard	Pass
2	Same call with wrong password	email: student@uni.edu password: wrongpass	Displays "Invalid Credentials"	Pass

Unit Testing 2: Group Creation

Testing Objective: Verify that a group is created only when all required details are provided and no group already exists for the student.

Table 18: Unit Testing 2

No.	Test Case	Attribute & Value	Expected Result	Result
1	GroupService.create()	groupName: "AI Vision", leaderID: 102	Group record inserted in database	Pass
2	Duplicate group attempt	leaderID: 102 again	Error: "Group already exists"	Pass

Unit Testing 3: Document Submission

Testing Objective: Ensure the portal accepts allowed file types and stores metadata correctly.

Table 19: Unit Testing 3

No.	Test Case	Attribute & Value	Expected Result	Result
1	SubmissionService.upload()	file: report.pdf	File stored; metadata saved	Pass
2	Unsupported file type	file: project.exe	Error: "Invalid File Type"	Pass

Unit Testing 4: Comment/Feedback Posting

Testing Objective: Ensure supervisors and students can post meaningful comments and system rejects empty messages.

Table 20: Unit Testing 4

No.	Test Case	Attribute & Value	Expected Result	Result
1	FeedbackService.add()	text: "Revise Chapter 2"	Saved; visible in thread	Pass
2	Empty text	text: ""	Error: "Comment cannot be empty"	Pass

Unit Testing 5: Notification Trigger

Testing Objective: Verify notifications are generated only when relevant actions occur.

Table 21: Unit Testing 5

No.	Test Case	Attribute & Value	Expected Result	Result
1	new submission uploaded	submissionID: 45	Supervisor receives notification	Pass
2	user refreshes without new events	N/A	No new notification	Pass

6.1.3 Functional Testing

Functional testing ensures that every feature performs according to the system requirements. Each module was tested by simulating real user actions for all roles, Students, Supervisors, HOD, and Admin.

Functional Testing 1: Multi-Role Login

Objective: Confirm that each user sees the correct dashboard based on their role.

Table 22: Functional Testing 1

No.	Test Case	Attribute & Value	Expected Result	Result
1	Login as Student	role = student	Student Dashboard with Group & Submission options	Pass
2	Login as Supervisor	role = supervisor	Supervisor Dashboard with Pending Requests & Submissions	Pass

3	Login as Admin	role = admin	Admin Dashboard with User Controls	Pass

Functional Testing 2: Group & Supervisor Assignment Workflow

Objective: Verify students can form a group, send supervisor requests, and supervisors can accept/reject.

Table 23: Functional Testing 2

No.	Test Case	Attribute & Value	Expected Result	Result
1	Student creates group	groupName: "Web Portal"	Group created	Pass
2	Send supervisor request	supervisorID: 20	Request visible in supervisor dashboard	Pass
3	Supervisor accepts request	status: approved	Group assigned to supervisor	Pass

Functional Testing 3: Document Submission & Evaluation

Objective: Ensure the upload-review-feedback cycle functions properly.

Table 24: Functional Testing 3

No.	Test Case	Attribute & Value	Expected Result	Result
1	Student uploads report	file: FYP.docx	Saved; status "Pending Review"	Pass
2	Supervisor reviews	feedback: "Add diagrams"	Status updated; feedback visible	Pass

Functional Testing 4: Search & Filtering

Objective: Ensure users can search submissions, groups, or announcements correctly.

Table 25: Functional Testing 4

No.	Test Case	Attribute & Value	Expected Result	Result
1	Search group	query: "AI"	Returns groups containing AI	Pass
2	Filter by supervisor	supervisor: "Dr. Ali"	Shows only assigned groups	Pass

6.1.4 Integration Testing

Integration testing ensures different components, database, backend logic, notification engine, messaging module, work together as a workflow.

Table 26: Integration Testing

No.	Test Case	Attribute & Value	Expected Result	Result
1	Student uploads document → Supervisor reviews → Student notified	file: report.pdf	Submission stored → Review saved → Notification delivered	Pass
2	Admin updates user role → Permissions refreshed	userID=200, newRole=supervisor	Dashboard updates; restricted areas unlocked	Pass
3	HOD posts announcement → Students receive	announcement="Submit FYP Draft"	Shown on dashboard + notification	Pass

6.2 Automated Testing

Automated testing was used to simulate repeated actions such as login, form submission, and permission checks. Browser automation tools were used to test UI behaviour and backend responses efficiently.

Table 27: Automated Testing

Tool Name	Tool Description	Applied On	Results
Selenium WebDriver	Automated browser testing (Chrome/Firefox)	Login, Group Creation, Supervisor Assignment workflows	Pass
Jest / Mocha	Automated backend testing for controllers & services	User Management, Submission Module, Notification Engine	Pass

Chapter 7 Conclusion and Future Work

7 Conclusion and Future Work

This chapter summarizes the overall outcomes of the FYP Portal project and evaluates how effectively the proposed system fulfills its objectives. It also highlights potential enhancements and future improvements that can further extend the system's functionality and institutional impact.

7.1 Conclusion

The development of the FYP Portal marks a significant advancement in digitalizing academic project management processes within educational institutions. Traditionally, the handling of Final Year Projects involved scattered communication, manual documentation, and time-consuming administrative coordination. By introducing a centralized, web-based system, the FYP Portal eliminates these inefficiencies and provides a structured, transparent, and user-friendly environment for all stakeholders involved in the FYP lifecycle.

The portal ensures that students can seamlessly form groups, request supervisors, submit their deliverables, and monitor their progress through an organized dashboard. This improved workflow reduces confusion, minimizes dependency on manual follow-ups, and ensures that all academic guidelines are adhered to in a timely manner. Supervisors benefit from a unified interface that allows them to manage multiple project groups, review submissions, provide feedback, and approve or reject requests efficiently. This structured system not only reduces administrative workload but also ensures fairness and consistency in evaluations.

From a departmental perspective, the HOD gains complete oversight of all ongoing projects, supervisor loads, deadlines, and academic activities through a consolidated dashboard. This visibility enhances decision-making and helps maintain academic quality assurance. Administrators play a crucial role by managing user accounts, defining system rules, and ensuring that the portal remains consistent with academic policies. Role-based access control ensures that every user interacts only with the functionalities relevant to their position, enhancing both security and usability.

Extensive manual and automated testing validated the robustness and reliability of the system. All major workflows, such as login authentication, group creation, supervisor allocation, file uploading, evaluation tracking, and notification delivery, performed consistently under different scenarios. Integration testing further confirmed that the portal components communicate effectively, ensuring that student submissions, supervisor actions, and administrative operations are synchronized accurately within the database.

The FYP Portal demonstrates strong performance characteristics, including quick load times, secure data handling, responsiveness across different devices, and intuitive navigation. The interface design prioritizes clarity and accessibility, ensuring ease of use even for non-technical users. In addition to supporting academic operations, the portal enhances transparency and accountability by maintaining a structured record of all actions, making it easier to track project history, monitor communication, and enforce deadlines.

Overall, the FYP Portal succeeds in modernizing the FYP management process by reducing delays, preventing data loss, improving communication, and enabling academic departments to manage large numbers of projects with efficiency and confidence. It establishes a scalable digital foundation that can be expanded and upgraded over time, ensuring long-term

sustainability and relevance in academic environments that continue to move toward digital transformation.

7.2 Future Work

While the FYP Portal successfully streamlines the management of Final Year Projects and addresses major pain points in traditional academic workflows, there are several opportunities for enhancement and expansion. These improvements can significantly increase usability, automation, and institutional integration, enabling the portal to evolve into a comprehensive academic project management ecosystem.

- **Web Portal Expansion with Advanced Dashboards:** A major extension involves enhancing the portal with advanced dashboards for students, supervisors, and departmental administrators. Future versions may include graphical analytics, project progress visualizations, supervisor workload charts, and automated alerts for overdue submissions. These improvements will offer richer insights and support data-driven academic decisions.
- **Automated Evaluation and Plagiarism Checking:** To further increase academic integrity and efficiency, the portal can integrate plagiarism detection APIs and automated evaluation tools. By incorporating similarity-checking systems, supervisors and departments can evaluate project originality more effectively. Automated rubric-based evaluation could also help generate preliminary assessment feedback before supervisor review.
- **AI-Powered Supervisor Recommendation System:** An intelligent recommendation engine can be added to match students or groups with supervisors based on research interests, workload balance, and previous supervision history. This feature would reduce manual allocation efforts and ensure a more equitable distribution of projects across faculty members.
- **Improved Communication and Collaboration Tools:** Future updates may include built-in real-time chat, video conferencing integration, group discussion threads, and collaborative document editing. These features will reduce reliance on external platforms and centralize all communication within the portal, leading to better documentation and smoother coordination between students and supervisors.
- **Enhanced Document Management and Version Control:** The portal can be extended to support version tracking of uploaded documents, automatic backup, and structured document comparison. This ensures that supervisors can review past versions of submissions, track improvements, and evaluate student progress more accurately.
- **Mobile Application Development:** To support wider accessibility, a dedicated mobile application for Android and iOS can be developed. This would allow students, supervisors, and administrators to receive real-time notifications, upload documents, review submissions, and manage tasks on the go.

8 References

[1] P. B. Faez, N. A. Abd Rahman, and K. S. Harun, "Online Project and Assignment Submission, Management and Progress Monitoring System (OPAS)," *Asia Pacific University of Technology and Innovation, Kuala Lumpur*, 2014.

[2] H. O. Salami and E. Y. Mamman, "A genetic algorithm for allocating project supervisors to students," *International Journal of Intelligent Systems and Applications*, vol. 8, no. 10, p. 51, 2016.

[3] S.-D. Kwon, M.-S. Lee, and D.-J. Cho, "Implementation of Web based Project Management System," in *Proceedings of the KAIS Fall Conference*, 2010, pp. 180–183.

[4] R. S. Pressman, *Software engineering: a practitioner's approach*. Palgrave macmillan, 2005.

[5] E. S. Walia and E. S. K. Gill, "A framework for web based student record management system using PHP," *International Journal of Computer Science and Mobile Computing*, vol. 3, no. 8, pp. 24–33, 2014.

[6] M. A. Bakar, N. Jailani, Z. Shukur, and N. F. M. Yatim, "Final year supervision management system as a tool for monitoring Computer Science projects," *Procedia-Social and Behavioral Sciences*, vol. 18, pp. 273–281, 2011.

[7] A. Ademola, A. Adewale, and D. U. Ike, "Design and development of a University portal for the management of final year undergraduate projects," *International Journal of Engineering and Computer Science*, vol. 2, no. 10, pp. 2911–2920, 2013.

[8] F. Hartman and R. A. Ashrafi, "Project management in the information systems and information technologies industries," *Project management journal*, vol. 33, no. 3, pp. 5–15, 2002.

[9] R. Wonohoesodo and Z. Tari, "A role based access control for web services," in *IEEE International Conference onServices Computing, 2004.(SCC 2004). Proceedings. 2004*, 2004, pp. 49–56.

[10] S. R. Bharamagoudar, R. B. Geeta, and S. G. Totad, "Web based student information management system," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 2, no. 6, pp. 2342–2348, 2013.

[11] P. Nitithamyong and M. J. Skibniewski, "Web-based construction project management systems: how to make them successful?," *Autom Constr*, vol. 13, no. 4, pp. 491–506, 2004.